7.8 Write a DTD for the document described in Exercise 7.7 with the following restrictions: the name, Social Security number, age, room number, and primary insurance company are required. All the other elements are optional, as are middle names.

7.9 Create a CSS style sheet for the XML document of Exercise 7.7 and use it to create a display of that document.

7.10 Create an XSLT style sheet for one `patient` element of the XML document of Exercise 7.7 and use it to create a display of that element.

7.11 Modify the XSLT style sheet of Exercise 7.6 to format all the `patient` elements in the XML document of Exercise 7.7 and use it to create a display of the whole document.

CHAPTER

8

The Basics of Perl

8.1 Origins and Uses of Perl
8.2 Scalars and Their Operations
8.3 Assignment Statements and Simple Input and Output
8.4 Control Statements
8.5 Fundamentals of Arrays
8.6 Hashes
8.7 References
8.8 Functions
8.9 Pattern Matching
8.10 File Input and Output
8.11 An Example
Summary ◦ *Review Questions* ◦ *Exercises*

Our primary interest in Perl in this book is its widespread use for Common Gateway Interface (CGI) programming. However, it is beneficial to most programmers to become familiar with Perl's capabilities. Perl is a flexible, powerful, widely used programming language—and it would be so even if CGI programming did not exist. This chapter takes you on a quick tour of Perl, introducing most of the important concepts and constructs but leaving out many of the details of the language. In spite of its brevity, however, if you are an experienced programmer, you can learn to write useful Perl programs by studying this chapter. The similarity of Perl to other common programming languages, especially C and JavaScript, makes this relatively easy, at least for those who know one of those languages. If you need more details, there are numerous books dedicated to Perl.

This chapter begins with a description of Perl's scalar values and variables and their use in expressions and assignment statements. Next, it covers control expressions and the collection of control statements available in Perl. Then it introduces Perl's two built-in data structures, arrays and hashes, and references. This is followed by a description of functions and how they are defined and called, including the peculiar way (for a high-level language) that parameters are passed. Finally, the chapter covers Perl's pattern matching and the basic operations of file input and output. Although we attempt to describe Perl in a single chapter, do not be misled into thinking that this is a small or simple language—it is neither.

8.1 Origins and Uses of Perl

Perl began as a relatively small language with the modest purpose of including and expanding on the operations of the text-processing language `awk` and the system administration capabilities of the UNIX shell languages, initially `sh`. Perl was first released in 1987 after being developed and implemented by Larry Wall. Since then it has grown considerably, borrowing features from other languages as well as inventing a few of its own.[1] Along the way, it picked up support for communications using sockets, a module construct, and support for object-oriented programming. Its text pattern-matching capabilities are elaborate and powerful, which is one of the reasons it became the most popular language for CGI programming. Perl's pattern matching has been copied into several other languages, including JavaScript, Ruby, and PHP. Perl is now used for many of the small- to medium-size programming projects formerly done in C.

Perl is a language whose implementation is between compiled (to machine code) and interpreted languages. Perl programs are compiled to an intermediate form, in part to check for errors, but mostly to make possible impressive run-time performance even though it is interpreted. When we refer to `perl`, we mean the Perl language processing system, which compiles and interprets programs. The `perl` system includes a debugger, among other things.

Perl has been ported to every common computing platform, from the various versions of UNIX to Windows and everything in between. Most versions of UNIX come with Perl, and Mac OS X has Perl installed.

8.2 Scalars and Their Operations

Perl has three categories of variables—scalars, arrays, and hashes—each identified by the first character of their names (`$` for scalar variables, `@` for array variables, and `%` for hash variables). This section discusses the important characteristics of the most commonly used kind of variables, namely, scalars. Scalar variables can store three different kinds of values: numbers, character strings, and references, which are addresses. We postpone discussing references until Section 8.7.

1. See `http://history.perl.org/` for more on the evolution of Perl.

The numeric values stored in scalar variables are represented internally in double-precision floating-point form. Although there is a way to suggest (to the compiler) that integer operations be used on scalar numeric values, in most cases the operations are done in double-precision floating point.

8.2.1 Numeric and String Literals

Numeric literals can have the forms of either integers or floating-point values. Integer literals are strings of digits. Floating-point literals can have either decimal points or exponents or both. Exponents are specified with an uppercase or lowercase `e` and a possibly signed integer literal. The following are legal numeric literals:

```
72  7.2  .72  72.  7E2  7e2  .7e2  7.e2  7.2E-2
```

Integer literals can be written in hexadecimal (base 16) by preceding their first digit with either `0x` or `0X`.

String literals can appear in two forms, depending on whether their delimiters are single quotes (`'`) or double quotes (`"`). Single-quoted string literals cannot include characters specified with escape sequences, such as newline characters specified with `\n`.[2] If an actual single-quote character is needed in a string literal that is delimited by single quotes, the embedded single quote is preceded by a backslash, as shown in the following example:

```
'You\'re the most freckly person I\'ve ever met'
```

If an escape sequence is embedded in a single-quoted string literal, each character in the sequence is taken literally as itself. For example, the sequence `\n` in the following string literal will be treated as two characters, a backslash and an `n`:

```
'You have freckles, \n but I don\'t'
```

If a string literal with the same characteristics as single-quoted strings is needed but you want to use a different delimiter, precede the delimiter with `q`, as shown in the following example:

```
q$I don't want to go, I can't go, I won't go!$
```

If the new delimiter is a parenthesis, a brace, a bracket, or an angle bracket, the left element of the pair must be used on the left, and the right element must be used on the right. For example:

```
q<I don't want to go, I can't go, I won't go!>
```

Double-quoted string literals differ from single-quoted string literals in two ways: First, they can include special characters specified with escape sequences; second, embedded variable names are interpolated into the string, which means that their values are substituted for their names. We discuss the first of these differences here; the other will be discussed in Section 8.2.2.

---

2. An *escape sequence* is one or two special characters followed by another character. The special character or characters change the meaning of the following character for that single appearance. For example, in some contexts, `\n` means a newline character, not a backslash followed by the letter *n*.

In many situations, we want to include special characters that are specified with escape sequences in string literals. For example, if we want the words on a line to be spaced by tabs, we use a double-quoted literal with embedded escape sequences for the tab character:

```
"Completion % \t Yards \t Touchdowns \t Interceptions"
```

A double quote can be embedded in a double-quoted string literal by preceding it with a backslash.

A different delimiter can be specified for string literals with the characteristics of double-quoted strings by preceding the new delimiter with `qq`:

```
qq@"Why, I never!", said she.@
```

The null string (one with no characters) can be expressed as `''` or `""`.

8.2.2 Scalar Variables

The names of all scalar variables, whether predefined or programmer defined, begin with dollar signs (`$`). The part of a scalar variable's name that follows the dollar sign is similar to the names of variables in other programming languages. It begins with a letter, which can be followed by any number of letters, digits, or underscore characters. There is no limit to the length of a variable name, and all of its characters are significant.[3] The letters in a variable name are case sensitive, meaning that `$FRIZZY`, `$Frizzy`, `$FrIzZy`, and `$friZZy` are all distinct names, although using all four of these in the same program would be frowned upon by most sensible people. However, by convention, programmer-defined variable names do not include uppercase letters.

As mentioned earlier, double-quoted string literals that contain the names of variables have the values of those variables included in, or interpolated into, the string. Consider the following string literal:

```
"Jack is $age years old"
```

If the value of `$age` is `47`, this string has the following value:

```
"Jack is 47 years old"
```

To place a variable name in a double-quoted string but not have it interpolated, the variable's name is preceded by a backslash. For example:

```
"The variable with the result is \$answer"
```

In Perl, variables often are not explicitly declared; the compiler declares a variable implicitly when it first encounters the variable's name in a program.[4]

A scalar variable that has not been assigned a value by the program has the value `undef`. The numeric value of `undef` is `0`; the string value of `undef` is the null string (`""`).

---

3. Well, at least no practical limit. Actually, a name can have no more than 255 characters.

4. Because a variable's name clarifies its type, explicit variable declarations would serve little purpose, except in functions (see Section 8.8).

Perl includes a large number of predefined, or *implicit*, variables. The names of implicit scalar variables begin with dollar signs. The rest of the name of an implicit variable is often just one more special character, such as an underscore (_), a circumflex (^), or a backslash (\). You will see many uses of these implicit variables in this chapter and the next.

The Perl software system includes extensive documentation. Specific parts of this documentation can be retrieved with the command `perldoc` followed by a topic. For example, a list of all Perl predefined variables can be found by typing the following:

```
perldoc perlvar
```

8.2.3 Numeric Operators

Most of Perl's numeric operators are similar to those in other common programming languages, so they should be familiar to most readers. They are the binary operators + for addition, - for subtraction, * for multiplication, / for division, ** for exponentiation, and % for modulus.[5] In addition, there are the unary operators for addition, subtraction, decrement (--), and increment (++). The decrement and increment operators can be either prefix or postfix.

Except under unusual circumstances, numeric operations are done in double-precision floating point. So, whereas the expression `5 / 2` may evaluate to `2` in many other languages, in Perl it evaluates to `2.5`.

The precedence rules of a language specify which operator is evaluated first when two operators that have different levels of precedence appear in an expression, separated only by an operand. The associativity rules of a language specify which operator is evaluated first when two operators with the same precedence level appear in an expression, separated only by an operand. The precedence and associativity of the numeric operators are given in Table 8.1.

Table 8.1 Precedence and associativity of the numeric operators

Operator	Associativity
++, --	Nonassociative*
unary +, -	Right
**	Right
*, /, %	Left
binary +, -	Left

The operators listed first have the highest precedence.
*An operator is nonassociative if two of them cannot appear in an expression separated only by an operand.

5. `$x % $y` produces the remainder of the value of `$x` after division by `$y`.

8.2.4 String Operators

Perl strings are not stored or treated as arrays of characters; rather, a string is a single unit. The two string operators are described in this section. The most commonly used string functions are introduced in Section 8.2.5.

String catenation is specified with the operator denoted by a period. For example, if the value of `$first` is `"Freddie"`, the value of the expression

```
$first . " Freeloader"
```

is

```
"Freddie Freeloader"
```

The repetition operator is specified with an `x`. It takes a string as its left operand and an expression that evaluates to a number as its right operand. The left operand is replicated the number of times equal to the value of the right operand. For example, the value of

```
"More! " x 3
```

is

```
"More! More! More! "
```

8.2.5 String Functions

Functions and operators in Perl are closely related. In fact, in many cases they can be used interchangeably. For example, if there is a predefined unary operator named `doit` that takes one parameter, it can be treated as an operator, as shown in the following example:

```
doit x
```

Or it could be treated as a function, as shown in the following:

```
doit(x)
```

A function with no parameters can be called with empty parentheses or no parentheses at all.

Table 8.2 lists the most commonly used string functions.

When an operator or function that expects a numeric operand is given a string, the string is implicitly converted (coerced) to a number. If the string does not represent a number, zero is used. When an operator or function that expects a string operand is given a number, the number is coerced to a string.

8.3 Assignment Statements and Simple Input and Output

Among the most fundamental constructs in most programming languages are assignment statements and the statements or functions that provide keyboard input and screen output. The next subsections introduce these as they appear in Perl.

Table 8.2 String functions

Name	Parameter(s)	Actions
`chomp`	A string	Removes any terminating newline characters* from its parameter; returns the number of removed characters
`length`	A string	Returns the number of characters in its parameter string
`lc`	A string	Returns its parameter string with all uppercase letters converted to lowercase
`uc`	A string	Returns its parameter string with all lowercase letters converted to uppercase
`hex`	A string	Returns the decimal value of the hexadecimal number in its parameter string
`join`	A character and the strings catenated together with a list of strings	Returns a string constructed by catenating the strings of the second and subsequent strings together, with the parameter character inserted between them

*The newline character may actually be two characters, as is the case with Windows.

8.3.1 Assignment Statements

Perl's assignment statements are the same as those of C and its descendants. The simple assignment operator is =, used in the following example:

```
$salary = 47500;
```

Compound assignment operators are binary operators with the simple assignment operator catenated to their right side. For example, the statement

```
$sum += $value;
```

is the same as this statement:

```
$sum = $sum + $value;
```

Notice that these sample assignment statements are terminated by semicolons. All Perl statements except those at the end of blocks (see Section 8.4) must be terminated by semicolons.

Comments in Perl are specified using the pound sign (#). Any text following a # on a line is ignored by the compiler (unless the # is in a literal string).

8.3.2 Keyboard Input

All input and output in Perl is uniformly thought of as file input and output. Files have external names but are referenced in programs through internal names, called *filehandles*. There are three predefined filehandles: `STDIN`, which is a program's normal input stream (by default, the keyboard); `STDOUT`, which is a program's normal output stream (by default, the screen); and `STDERR`, which is

a program's normal output stream for error messages (usually also associated with the screen).

The line input operator[6] is different from other operators. It is specified with a pair of angle brackets <>, with its operand, if one is provided, embedded between the brackets. For example, you get a line from `STDIN` as follows:

```
$in_data = <STDIN>;
```

The line input operator gets all characters typed on the keyboard up to and including the newline character. In many cases, the newline character is not wanted, so the following idiom is common:

```
chomp($in_data = <STDIN>);
```

8.3.3 Screen Output

Output is directed to the screen with the `print` function (or operator). We prefer to treat `print` as an operator. The operand for `print` is one or more string literals, separated by commas. No implicit newline character is appended to the last string operand, so if one is needed, it must be included, as shown in the following:

```
print "This is pretty easy \n";
```

Many screen output lines include the value of one or more program variables. Because the variable names in a double-quoted string are interpolated, this is easy. The `printf` function from C is also available in Perl, including format codes such as `%7d` and `%5s`.

The following trivial program illustrates some of what we have discussed so far:

```
# quadeval.pl - A simple Perl program
#  Input: Four numbers, representing the values of
#       a, b, c, and x
# Output: The value of the expression
#       axx + bx + c
# Get input
print "Please input the value of a ";
$a = <STDIN>;
print "Please input the value of b ";
$b = <STDIN>;
print "Please input the value of c ";
$c = <STDIN>;
```

6. Note that the line input operator is also used to get input from places other than the keyboard, as we explain later. The line input operator is sometimes called the *angle operator*; it is also sometimes called the *diamond operator*.

```
print "Please input the value of x ";
$x = <STDIN>;
# Compute and display the result
$result = $a * $x * $x + $b * $x + $c;
print "The value of the expression is: $result \n";
```

Under Windows and UNIX, a Perl program can be run from the operating system command-line prompt, by typing `perl` followed by the filename where the Perl program is stored, as shown in the following:

```
perl quadeval.pl
```

This executes the program named `perl`, which is the Perl compiler/interpreter. It causes the program in the file `quadeval.pl` to be compiled and interpreted. If you want a compilation without the interpretation, just to check the syntactic correctness of your program, include the `-c` flag after the `perl` command. It is always a good idea to include the `-w` flag, which causes `perl` to produce warning messages for a variety of suspicious things it may find in your program. This is useful because Perl is a very forgiving language.

The line input operator can be used to get input from a file specified as a command-line argument, which can appear at the end of the `perl` command. The operator in this case is empty angle brackets. For example, if `perl` is run with the following command:

```
perl –w quadeval.pl quad.dat
```

then the following statement puts the first line from the file `quad.dat` into `$input`:

```
$input = <>;
```

An alternative way to run Perl programs is discussed in Chapter 9, "Using Perl for CGI Programming."

8.4 Control Statements

Perl has a powerful collection of statements for controlling the execution flow through its programs. This section introduces the control expressions and control statements of Perl.

8.4.1 Control Expressions

The expressions upon which statement control flow is based are either scalar-valued expressions, relational expressions, or compound expressions. If the value of a scalar-valued expression is a string, it is true unless it is either the empty

string ("") or a zero string ("0").[7] If the value is a number, it is true unless it is zero (0).

Relational operators can have any scalar-valued expression for their operands. Perl has two sets of relational operators, one for numeric operands and one for string operands. Table 8.3 lists the relational operators.

Table 8.3 Relational operators

Operation	Numeric Operands	String Operands
Is equal to	==	eq
Is not equal to	!=	ne
Is less than	<	lt
Is greater than	>	gt
Is less than or equal to	<=	le
Is greater than or equal to	>=	ge
Compare, returning -1, 0, or +1	<=>	cmp

If a string relational operator is given a numeric operand, the value of that operand is coerced to a string. Likewise, the numeric operators coerce string operands to numbers. In some cases, the value produced by a coercion may not be intuitive. For example, when the string 'George' is used as an operand of >, it is coerced to zero, because zero is produced whenever the string cannot be coerced to a number. Coercions do not produce error or warning messages when an operand must be coerced, even if the coercion is required only because the programmer typed the wrong operand. These coercions are obviously dangerous because they prevent the system from detecting some programmer errors, so be careful to use the relational operator that is appropriate for the operands.

The first six relational operators in Table 8.3 produce +1 if true, "" if false.

The <=> and cmp operators compare their operands and produce -1 if the left operand is less than the right operand, 0 if they are equal, and +1 if the left operand is greater than the right operand. These operators are useful with the sort operator, which is discussed in Section 8.8.4.

Perl has two sets of operators for the AND, OR, and NOT Boolean operations. These two sets have the same semantics but different precedence levels. The operators with the higher precedence level are && (AND), || (OR), and ! (NOT). Those with the lower precedence are and, or, and not. The precedence of these latter operators is lower than any other operators in Perl, so

7. This can be misinterpreted. The string "0.0" may look like a false value, but because it is not precisely "0", it is true.

regardless of what operators appear in their operands, these operators will be evaluated last.

The precedence and associativity of all operators discussed so far in this chapter are shown in Table 8.4.

Table 8.4 Operator precedence and associativity

Operator	Associativity
`++, --`	Nonassociative
`**`	Right
unary `+`, unary `-`	Right
`*, /, %, x`	Left
`+, -, .`	Left
`>, <, >=, <=, lt, gt, le, ge`	Nonassociative
`==, !=, <=>, eq, ne, cmp`	Nonassociative
`&&`	Left
`||`	Left
`=, +=, -=, *=, **=, /=, %=, &=, &&=, ||=, x=`	Right
`not`	Right
`and`	Left
`or`	Left

Highest-precedence operators are listed first.

Because assignment statements have values (the value of an assignment is the value assigned to the left-side variable), they can be used as control expressions. One common use of this is for an assignment statement that uses `<STDIN>` as its right side. The line input operator returns the empty string when it gets the end-of-file (EOF) character, so this can be conveniently used to terminate loops. For example:

```
while ($next = <STDIN>) { … }
```

The keyboard EOF character is Ctrl+D for UNIX, Ctrl+Z for Windows, and Cmd+. (period) for Macintosh systems.

8.4.2 Selection and Loop Statements

Control statements require some syntactic container for sequences of statements whose execution they are meant to control. Perl uses the block for this container. A block is formed by putting braces (`{}`) around a sequence of state-

ments. Blocks can have local variables, as you will see in Section 8.8. A *control construct* is a control statement and the block whose execution it controls.

Perl's `if` statement is similar to that of other languages. The only thing a bit different is that both the then clause and the else clause must always be blocks. For example, the following construct is illegal:

```
if ($a > 10)
    $b = $a * 2;  # Illegal – not a block
```

To be legal, the then clause must be a block, as shown in this example:

```
if ($a > 10) {
    $b = $a * 2;
}
```

An `if` construct can include `elsif` (note that it is *not* spelled "elseif") clauses, which provide a way of having a more readable sequence of nested `if` constructs. For example:

```
if ($snowrate < 1) {
    print "Light snow \n";
} elsif ($snowrate < 2) {
    print "Moderate snow \n";
} else {
    print "Heavy snow \n";
}
```

Perl has an `unless` statement, which is the same as its `if` statement except that the inverse of the value of the control expression is used. This is convenient if you want a selection construct with an else clause but no then clause. The following construct illustrates an `unless` statement:

```
unless ($sum > 1000) {
    print "We are not finished yet! \n";
}
```

The Perl `while` and `for` statements are similar to those of C and its descendants. The bodies of both must be blocks. The general form of the `while` statement is as follows:

`while` (*control expression*) {
  loop body statement(s)
}

The `until` statement is similar to the `while` statement except that the inverse of the value of the control expression is used.

Perl's `for` statement is most often used for loops controlled by counters. The general form of the `for` statement is as follows:

`for` (*initial expression*; *control expression*; *increment expression*) {
  loop body statement(s)
}

Both the initial expression and the increment expression can be multiple expressions, separated by commas, in which case the value of the whole expression is that of the expression following the last comma. The following `for` statement illustrates its use in forming a simple counter-controlled loop:

```
$sum = 0;
for ($counter = 1; $counter <= 100; $counter++) {
    $sum += $counter;
}
```

The operators `last` and `next` provide a way to exit a loop (or any block). These are exactly like the `break` and `continue` statements of C except they can include labels, which allow more than one loop or block to be exited. Consider the following skeletal example:

```
BIGLOOP:
while (...) {
    while (...) {
        ...
        if (...) {last BIGLOOP;}
        ...
    }
    ...
}
```

In this code, the `last` operator transfers control out of both loops.

Perl has no `switch` statement.

The implicit variable `$_` is frequently used in Perl programs, most often as the default parameter in a function call and as the default operand of an operator. It is also the default target for the input operator. For example, consider the following statement:

```
<STDIN>;
```

This statement gets a line from the keyboard and assigns it to `$_`.

The following example illustrates some of the uses of `$_`:

```
while (<STDIN>) {
    print;
    chomp;
    if ($_ eq "redhead") {
        print "I've finally found one! \n";
    }
}
```

There are three uses of `$_` in this `while` construct: as the target of the input line operator, as the default parameter to `print`, and as the default operand of `chomp`. As you might suspect, the heavy use of `$_`, especially when it is only implied, is not highly regarded by some software developers who normally use other programming languages.

8.5 Fundamentals of Arrays

Arrays in Perl are more flexible than those of most of the other common languages. This is a result of three fundamental differences between Perl arrays and those of other common languages such as C, C++, and Java. First, the length of a Perl array is dynamic—it can grow or shrink any time during program execution. Second, Perl arrays can have absent elements. For example, a Perl array may have only three elements, but those elements may have the subscripts `10`, `100`, and `352`. Third, a Perl array can store different types of scalar data. For example, an array may have some numeric elements and some string elements.

8.5.1 List Literals

A *list* is an ordered sequence of scalar values. A *list literal*, which is a parenthesized list of scalar values, is the way a list value is specified in a program. Arrays store lists, so list literals serve as array literals. Because each list element can be any kind of scalar value, a list literal can be any combination of them. An expression can also be used to describe an element of a list literal. For example:

```
(3.14149 * $radius, "circles", 17)
```

8.5.2 Arrays

We use the term *array* to denote a variable that stores lists. All array names begin with an at sign (`@`), which puts them in a namespace that is different from that of the scalar variable names. Arrays can be assigned list literals or other arrays. Consider the following examples:

```
@list =  ('boy', 'girl', 'dog', 'cat');
@creatures = @list;
```

When an array is assigned to another array, as in the second assignment statement above, a new array is created for the target variable (`@creatures` in the example). This is different from C, where array names without subscripts are treated as pointers, and an array assignment such as the preceding example would result in the target variable being set to the address of the assigned array. In Perl, array names without subscripts are never treated as pointers.

The context of an assignment statement depends on the type of the target variable—if it is a scalar, the context is scalar; if it is an array or a list, it is called list context. If an array is used in scalar context, the array's length (the number of elements that are in the array) is used. For example, the following statement assigns `4` to `$len` because there are four elements in (the previously defined) `@list`:

```
$len = @list;
```

This is a rather odd way to get the length of an array. It is important to remember that it is easy to put an array in scalar context by mistake, and Perl will not detect it as an error.

A list literal that contains only scalar variable names can be the target of a list assignment:

```
($first, $middle, $last) = ("George", "Bernard", "Shaw");
```

Such an assignment is called a *list assignment*. In the example, list assignment is used as shorthand for the following:

```
$first = "George";
$second = "Bernard";
$third = "Shaw";
```

If the target of a list assignment includes an array, all remaining values on the right side go to the array. Therefore, if an array is in the target of a list assignment, it should appear last.

All Perl array elements use integers as subscripts, and the lower-bound subscript of every array is zero. Array elements are referenced through subscripts delimited by brackets (`[ ]`). A subscript can be any numeric-valued expression. Because an array element is always a scalar, the scalar version of the array's name is used when a subscript is attached:

```
@list = (2, 4, 6, 8);
$second = $list[1];  # Sets $second to 4
```

Though it makes some sense, this use of a scalar name to reference an element of an array is confusing to most beginning Perl programmers.

It is essential to remember that scalar and array variables are in distinct namespaces. When a scalar name is followed by a subscript, it is no longer in the scalar namespace—the subscript moves it to the array namespace. Therefore, there is no connection between `$a` and `$a[5]`.

The length of an array is dynamic; the highest subscript to which a value has been assigned determines the current length of the array:

```
@list = ("Monday", "Tuesday", "Wednesday", "Thursday");
$list[4] = "Friday";
```

Here, the length of `@list` is now 5. Note that the length of an array is determined by the highest used subscript, which may be unrelated to the number of elements in the array.

Consider the following code:

```
@list = (2, 4, 6);
$list[27] = 8;
```

Now `@list` has four elements, but its length is 28. It has 24 vacant positions, which do not have elements.

The last subscript of `@list` can be referenced as `$#list`. So, the length of `@list` is `$#list + 1`. The last subscript of an array can be assigned to set its length to whatever you want, as shown in the following example:

```
$#list = 999;
```

As discussed previously, two different contexts of a variable name or expression exist: scalar and list. An expression assigned to a scalar variable is in scalar context; an expression assigned to an array or list is in list context. Some of Perl's operators force either scalar or list context on their operands. Likewise, some functions force either scalar or list context on their parameters. Scalar context can be forced with the pseudo function `scalar`, as shown here:

```
scalar(@list)
```

There is no way to force list context on an expression.

8.5.3 The foreach Statement

The `foreach` statement is used to process the elements of an array. For example, the following code will divide all of the values in `@list` by 2:

```
foreach $value (@list) {
    $value /= 2;
}
```

The scalar variable in a `foreach` becomes an alias (a new name) for each of the array's elements, one at a time. So, although the assignment statement in this `foreach` body appears to be changing the scalar variable `$value`, it is in fact changing the values of the elements of the array `@list`.

The scalar variable that appears in the `foreach` statement is local to the `foreach` construct. So, if the program has another variable with the same name, the one in the `foreach` will not interfere with the one used outside the construct.

If the array specified in a `foreach` statement has vacant spaces, `foreach` behaves as if the vacant elements existed and had the value `""`. For example, suppose an array is defined as follows:

```
$list[1] = 17;
$list[3] = 34;
```

Now consider the following loop:

```
foreach $value (@list) {
    print "Next: $value \n";
}
```

This loop produces the following:

```
Next:
Next: 17
Next:
Next: 34
```

8.5.4 Built-In Array Functions

As with other Perl functions, these can be treated as either operators or functions—operators if the parameters are not parenthesized, functions otherwise. It is common to need to place new elements on one end or the other of an array. Perl has four functions for these purposes: `unshift` and `shift`, which deal with the left end of arrays (lowest subscript); and `pop` and `push`, which deal with the right end of arrays (highest subscript).

The `shift` function removes and returns the first element of its given array parameter. For example, the following statement removes the first element of `@list` and places it in `$first`:

```
$first = shift @list;
```

The subscripts of all of the other elements in the array are reduced by 1 as a result of the `shift` operation. The `pop` function removes and returns the last element of its given array operand. In this case, there is no change in the subscripts of the array's other elements.

The `unshift` function takes two parameters: an array and a scalar or list. The scalar or list is appended to the beginning of the array. This results in an increase in the subscripts of all other array elements. The `push` function also takes an array and a scalar or a list. The scalar or list is added to the high end of the array:

```
@list = (2, 4, 6);
push @list, (8, 10);
```

The value of `@list` is now `(2, 4, 6, 8, 10)`.

Either `pop` and `unshift` or `push` and `shift` can be used to implement a queue in an array, depending on the direction the queue is intended to grow.

8.5.5 Built-In List Functions

List functions take lists as parameters. Because lists are immutable, these functions cannot change their list parameters. Instead, they yield new lists.

The `split` function is used to break strings into parts using a specified character as the basis for the split. The resulting substrings are placed in a specified array, as shown in the following example:

```
$stoogestring = "Curly Larry Moe";
@stooges = split(" ", $stoogestring);
```

The three elements of `@stooges` are now `"Curly"`, `"Larry"`, and `"Moe"`.

The `sort` function takes an array parameter and uses a string comparison to sort the elements of the array alphabetically in the returned list. For example:

```
@new_names_list = sort @names_list;
```

Note that `sort` does not alter its parameter, so in this statement, `@names_list` is unchanged.

Recall that if either operand of a string comparison (relational) operator is not a string, it is coerced to a string before the comparison operation takes place. It is possible to supply a comparison function to allow `sort` to sort all kinds of data. Do not use the `sort` operator to sort numbers if you do not supply a comparison function. If you do, `sort` will convert the numbers to strings, and you probably will not get what you expected. Section 8.8.4 discusses the use of the sort function for other orders and for nonstring array elements.

The `qw` function can be used on a sequence of unquoted strings to quote all of them, as shown in the following example:

```
qw(peaches apples pears kumquats)
```

This call to `qw` produces the following:

```
("peaches", "apples", "pears", "kumquats")
```

Notice that spaces, not commas, separate the list elements in the call to `qw`.

The `die` list operator is similar in form to the `print` function, which also can be considered a list operator. It takes a variable number of string parameters, catenates them, sends the result to `STDERR`, and terminates the program. The implicit variable `$!` stores the number of the most recent error that has occurred. It is useful for debugging to include `$!` in the parameter to `die`. For example:

```
die "Error -- division by zero in function fun2 - $!";
```

8.5.6 An Example

The following example illustrates a simple use of an array. A file of names, whose name is specified on the command line, is read. The names are converted to all uppercase letters, and the array is sorted and displayed.

```
# process_names.pl - A simple program to illustrate
#                    the use of arrays
#  Input: A file, specified on the command line, of
#         lines of text, where each line is a person's
#         name
# Output: The input names, after all letters are converted
#         to uppercase, in alphabetical order

$index = 0;

#>>> Loop to read the names and process them
while($name = <>) {

#>>> Convert the name's letters to uppercase and put it in
#>>>  the names array
    $names[$index++] = uc($name);
```

```
}

#>>> Display the sorted list of names
print "\nThe sorted list of names is:\n\n\n";
foreach $name (sort @names) {
    print ("$name \n");
}
```

8.6 Hashes

Associative arrays are arrays in which each data element is paired with a key, which is used to find the data element. Because hash functions are used to find specific elements in an associative array, Perl associative arrays are called *hashes*. The two fundamental differences between arrays and hashes are as follows: First, arrays use numeric subscripts to address specific elements, whereas hashes use string values (the keys) for element addressing. Second, the elements in arrays are ordered by subscript, but the elements in hashes are not. In a sense, elements of an array are like those in a list, whereas elements of a hash are like those in a set, where order is irrelevant. The actual arrangement of the elements of a hash in memory is determined by the internal hash function used to insert and access them.

Names of hash variables begin with percent signs (%), which places them in their own namespace. List literals may be used to initialize hash variables. The symbols => can be used between a key and its associated data element, or value, in a list literal used to initialize a hash variable. Commas can also be used, but they do nothing to connote the fact that it is a literal hash value:

```
%kids_ages = ("John" => 38, "Genny" => 36, "Jake" => 22,
              "Darcie" => 21);
```

Arrays also can be assigned to hashes, with the sensible semantics that the odd-subscripted elements of the array become the values of the hash, and the even-subscripted elements of the array become the keys of the hash. Hashes also can be assigned to arrays. When an array is assigned to a hash or a hash is assigned to an array, a copy process is used. Therefore, after an array is assigned to a hash, for example, subsequent changes to the hash do not affect the array.

An individual value element of a hash can be referenced by "subscripting" the hash name with a key. Braces are used to specify the subscripting operation. The name of a reference to a hash element begins, of course, with a dollar sign instead of the percent sign (because it is a scalar value):

```
$genny_age = $kids_ages{"Genny"};
```

New values are added to a hash by assigning the value of the new element to a reference to the key of the new element, as shown in the following example:

```
$kids_ages{"Aidan"} = 7;
```

An element is removed from a hash with the `delete` operator, as shown here:

```
delete $kids_ages{"Genny"};
```

A hash can be set to empty in two ways: First, an empty list can be assigned to the hash. Second, the `undef` operator can be used on the hash. These are illustrated with the following statements:

```
%kids_ages = ();
undef %kids_ages;
```

The `exists` operator is used to determine whether an element with a specific key is in a hash. For example:

```
if (exists $kids_ages{"Freddie"}) ..
```

The keys and values of a hash can be extracted into arrays with the operators `keys` and `values`, respectively:

```
foreach $child (keys %kids_ages) {
  print "The age of $child is $kids_ages{$child} \n";
}
@ages = values %kids_ages;
print "All of the ages are: @ages \n";
```

If a hash variable is embedded in a double-quoted string literal, its keys and values are not interpolated into the string. To display all of the keys and values of a hash, first assign it to an array and then print the array.

Perl has a predefined hash named `%ENV` that stores operating system environment variables. Environment variables are used to store information about the system on which `perl` is running. The environment variables and their respective values in `%ENV` can be accessed by any Perl program. The keys of `%ENV` are the names of the environment variables; the values are, of course, their values. All of the environment variables and their values of a specific system can be displayed with the following:

```
foreach $key (sort keys %ENV) {
    print "$key = $ENV{$key} \n";
}
```

In Chapter 9, "Using Perl for CGI Programming," we will make use of environment variables.

8.7 References

A *reference* is a scalar variable that references another variable or a literal. So, the value of a reference variable is an address. Although Perl's references are related to the pointers in C and C++, they are less flexible and much safer. The address

of an existing variable is obtained with the backslash operator on the name of that variable:

```
$age = 42;
$ref_age = \$age;
@stooges = ("Curly", "Larry", "Moe");
$ref_stooges = \@stooges;
```

A reference to a list literal can be created by putting the literal value in brackets, as follows:

```
$ref_salaries = [42500, 29800, 50000, 35250];
```

A reference to a hash literal is created by putting the literal value in braces:

```
$ref_ages = {
    'Curly' => 41,
    'Larry' => 38,
    'Moe' => 43,
};
```

A reference can specify two different values: its own, which is an address, or the value at that address. The process of making an appearance of a reference variable specify the latter is called *dereferencing*. All dereferencing in Perl is explicit. So, if you want the value to which a reference points rather than the address value of the reference, you must use different syntax on the reference's name. There are two ways to do this in Perl. First, an extra dollar sign can be appended to the beginning of the reference's name. For example, the value of `$$ref_age` is `42` rather than the address of `$age`.

If the reference is to an array or hash, there is a second way to specify dereferencing, which is to use the `->` operator between the variable's name and its subscript. For example, the following two assignment statements are equivalent:

```
$$ref_stooges[3] = "Maxine";
$ref_stooges -> [3] = "Maxine";
```

8.8 Functions

Subprograms are central to the usefulness of any programming language. Perl's subprograms are all functions, as in its ancestor language, C. This section describes the basics of Perl functions.

8.8.1 Fundamentals

A *function definition* includes the function's header and a block of code that describes its actions. Neither parameter specifications nor a return value type are part of a Perl function definition. A *function header* is the reserved word `sub` and the function's name. A *function declaration* is a message to the compiler that

the given name is a function that will be defined somewhere in the program. Syntactically, a function declaration is the function header without the block. Because forward calls to functions are legal, it does not matter where their definitions appear in a program. We prefer to put them at the beginning.

A function that returns a value that is to be used immediately is called in the position of an operand in an expression (or as the whole expression). A function that either does not return a value or returns a value that is not to be used can be called by a standalone statement.

A function that has been previously declared can be treated as a list operator, meaning that calls to it do not need to include the parentheses.

A function definition can specify the value it returns in two ways, implicitly and explicitly. The `return` function takes an expression as its parameter. The value of the expression is returned when the `return` is executed. A function can have any number of calls to `return`, including none. If there are no calls to `return` in a function or if execution arrives at the end of the function without encountering a `return`, its returned value is the value of the last expression evaluated in the function. For clarity's sake, we recommend that the last statement of every function is a `return`.

8.8.2 Local Variables

Variables that are implicitly declared have global scope—that is, they are visible in the entire program. It is usually best for variables used in a function that are not used outside the function to have local scope, meaning that they are visible and can be used only within the block of the function. Such variables are declared to have local scope in a function with a `my` declaration.[8] Initial values of local variables can be part of their declaration as follows:

```
my $count = 0;
```

If more than one variable is declared in a `my` declaration, they must be placed in parentheses, as shown in the following example:

```
my($count, $sum) = (0,0);
```

Notice the use of the list assignment to initialize these local variables.

If the name of a local variable conflicts with that of a global variable, the local variable is used. This is the advantage of local variables: When you make up their names, you do not need to be concerned that a global variable with the same name may exist in the program.

Perl includes a second kind of local variable, which is declared with the `local` reserved word. The scope of such variables is dynamic, which makes them very different from `my` variable, whose scope is static. We do not discuss this second kind of local variable here. When we say a variable is local, we mean it has local scope and is defined with `my`.

8. Actually, `my` is a function, but it seems clearer to consider it a declaration.

8.8.3 Parameters

The parameter values that appear in a call to a function are called *actual parameters*. The parameter names used in the function, which correspond to the actual parameters, are called *formal parameters*. Two common models of parameter transfers are used in the linkage between a function and its caller: pass by value and pass by reference. Pass-by-value parameters, which are usually implemented by sending values to the function, provide one-way communication to the called function. Pass-by-reference parameters, which are often implemented by passing the addresses of variables to the function, provide two-way communication between the function and its caller. Unless two-way communication is necessary, pass-by-value parameters should be used.

All Perl parameters are communicated through a special implicit array, `@_`. When a function is called, the values of the actual parameters specified in the call are placed in `@_`. If an actual parameter is an array, all of the array's elements are placed in `@_`. When a hash is used as an actual parameter, its value is flattened into an array, and its values are moved to `@_`. Every function has its own version of `@_`, so if a function calls another function, the values in the caller's `@_` are not affected by the call.

If the values in `@_` are manipulated directly in the called function, the parameters have pass-by-reference semantics, as shown in the following example:

```
sub plus10 {
    $_[0] += 10;
}
plus10($a);
```

The call to `plus10` results in `10` being added to `$a`. If you call this function with a literal—by mistake, of course—it has no effect, not even that of producing an error message.

Pass-by-value parameters are implemented by assigning the passed values in `@_` to local variables. For example:

```
sub fun_eval {
    my($a, $b, $c, $x) = @_;
    return $a * $x * $x + $b * $x + $c;
}
```

This function evaluates a given quadratic equation.

References to variables can be used as actual parameters, which provides pass-by-reference semantics. For example, the following function builds a new array from the nonnegative values from a given array.

```
sub squeeze {
    my $ref_list = $_[0];
    my $value, @new;
    foreach $value (@$ref_list) {
        if($value > 0) {
```

```
            push(@new, $value);
        }
    }
    return @new;
}
```

The following is an example of a call to `squeeze`:

```
squeeze(\@mylist);
```

8.8.4 The sort Function, Revisited

Recall that the `sort` function, as introduced in Section 8.5.5, takes a single array parameter and sorts that array, treating the elements as strings (coercing elements that are not strings to strings). This does not work for numbers because, when coerced to strings, `124` belongs before `2`.

The `sort` function can be used more flexibly by giving it the code to use to compare array elements. This code appears as a block between `sort` and the array parameter. Because the block is not a parameter, there is no comma between it and the array to be sorted. The comparison code block must evaluate to a negative number if the first value being compared belongs before the second, zero if the two values are equal, and a number greater than zero if the two values must be interchanged. The two values to be compared are referenced in the comparison with the names `$a` and `$b`. These variables, which act as formal parameters, are used in pass-by-reference mode. Therefore, they should not be changed in the comparison code. The two relational operators, `<=>` and `cmp`, are normally used for the comparison. Recall that they return exactly what is needed by the sort process.

For example, to sort numbers in the array `@list` into ascending order, the following could be used:

```
@new_list = sort {$a <=> $b} @list;
```

To sort the same array into descending order, the two variables `$a` and `$b` are interchanged in the comparison, as shown in the following:

```
@new_list = sort {$b <=> $a} @list;
```

Likewise, to sort strings in the array `@names` into reverse alphabetic order, the following could be used:

```
@new_names = sort {$b cmp $a} @names;
```

The comparison process for a `sort` can be defined in a function, in which case the function's name takes the place of the block in the call to `sort`.

8.8.5 An Example

What follows is an example of a program that uses a function to compute the median of a given array of numbers. The address of the array is passed to the function and is moved to a local variable there. There is no need to pass the

length of the array because the function can easily determine the array's length. The precise specification for the function is given in its initial comments.

```
# tst_median.pl - a program to test a function that
#                 computes the median of a given array

# median - a function
# Parameter:
#     A reference to an array of numbers
# Return value:
#     The median of the array, where median is the
#      middle element of the sorted array, if the
#      length is odd; if the length is even, the median
#      is the average of the two middle elements of the
#      sorted array

sub median {
    my $ref_list = $_[0];

#>>> Compute the length of the passed array
    my $len = $#$ref_list + 1;

#>>> Sort the parameter array
    @list = sort {$a <=> $b} @$ref_list;

#>>> Compute the median
    if ($len % 2 == 1) {  # length is odd
        return $list[$len / 2];
    } else {  # length is even
        return ($list[$len / 2] + $list[$len / 2 - 1]) / 2;
    }
}  #>>> End of function median

#>>> Begin main program
#>>> Create two test arrays, one with odd length and one with
#>>>  even length
@list1 = (11, 36, 5, 20, 41, 6, 8, 0, 9);
@list2 = (43, 77, 11, 29, 8, 51, 9, 18);

#>>> Call median on both arrays and display the results
$med = median(\@list1);
print "The median of the first array is: $med \n";
$med = median(\@list2);
print "The median of the second array is: $med \n";
```

The output of this program is as follows:

```
The median of the first array is: 9
The median of the second array is: 23.5
```

8.9 Pattern Matching

Regular expressions in JavaScript were discussed in Chapter 4, "The Basics of JavaScript." Because JavaScript regular expressions are based directly on those of Perl, readers who are not familiar with regular expressions are referred to Sections 4.12.1 to 4.12.3. The pattern-matching operations of Perl are different from those of JavaScript, so they are discussed here.

8.9.1 The Basics of Pattern Matching

In Perl, the pattern-matching operation is specified with the operator `m`. When slashes are used as delimiters, the `m` operator is not required. Therefore, just a slash-delimited regular expression is a complete pattern expression in Perl. The string against which the matching is attempted is, by default, the implicit variable `$_`. The result of evaluating a pattern-matching expression is either `true` or `false`. The following is a pattern match against the value in `$_`:

```
if (/rabbit/) {
    print
        "The word 'rabbit' appears somewhere in \$_ \n";
}
```

The pattern-matching operation does not need to be always against the string in `$_`. The binding operator, `=~`, can be used to specify any string as the one against which the pattern will be matched. For example, consider the following:

```
if ($str =~ /^rabbit/) {
    print "The value of \$str begins with 'rabbit' \n";
}
```

A restricted form of the `split` function was introduced in Section 8.5.5. In that section, the first parameter to `split` was a single character. However, the first parameter also can be any pattern. For example, we could have the following:

```
@words = split /[ .,]\s*/, $str;
```

This statement puts the words from `$str` into the `@words` array. The words in `$str` are defined to be terminated with either a space, a period, or a comma, any of which could be followed by more whitespace characters.

The following sample program illustrates a simple use of pattern matching and hashes. The program reads a file of text in which the words are separated by

whitespace and some common kinds of punctuation such as commas, periods, semicolons, and so forth. The objective of the program is to produce a frequency table of the words found in the input file. A hash is an ideal way to build the word-frequency table. The keys can be the words, and the values can be the number of times they have appeared. The `split` operator provides a convenient way to split each line of the input file into its words. For each word, the program uses `exists` on the hash to determine whether the word has occurred before. If so, its count is incremented; if not, the word is entered into the hash with a count of `1`.

```
# word_table.pl
#  Input: A file of text in which all words are separated by white-
#         space or punctuation, possibly followed by whitespace,
#         where the punctuation can be a comma, a semicolon, a
#         question mark an exclamation point, a period, or a colon.
#         The input file is specified on the command line
# Output: A list of all unique words in the input file,
#         in alphabetical order

#>>> Main loop to get and process lines of input text
while (<>) {

#>>> Split the line into words
    @line_words = split /[\.,;:!\?]\s*/;

#>>> Loop to count the words (either increment or initialize to 1)
    foreach $word (@line_words) {
        if (exists $freq{$word}) {
            $freq{$word}++;
        } else {
            $freq{$word} = 1;
        }
    }
}
#>>> Display the words and their frequencies
print "\n Word \t\t Frequency \n\n";
foreach $word (sort keys %freq) {
    print " $word \t\t $freq{$word} \n";
}
```

Notice that the two normally special characters, `.` (period) and `?` (question mark), are not backslashed in the pattern for `split` in this program. The reason is that, as mentioned previously, the normally special characters for patterns (metacharacters) are not special in character classes.

8.9.2 Remembering Matches

The part of the string that matched a part of the pattern can be saved in an implicit variable for later use. The part of the pattern whose match you want to save is placed in parentheses. The substring that matched the first parenthesized part of the pattern is saved in `$1`, the second in `$2`, and so forth. As an example, consider the following:

```
"4 July 1776" =~ /(\d+) (\w+) (\d+)/;
print "$2 $1, $3 \n";
```

This displays the following:

```
July 4, 1776
```

In some situations, it is convenient to be able to reference the parts of the string that preceded the match, the part that matched, or the part that followed the match. These three strings are available after a match through the implicit variables `$`` `, `$&`, and `$´`, respectively.

8.9.3 Substitutions

Sometimes the substring of a string that matched a pattern must be replaced by another string. Perl's substitute operator is designed to do exactly that. The general form of the substitute operator is as follows:

`s/`*Pattern*`/`*New_String*`/`

Pattern is the same as the patterns used by the match operator. *New_String* is what is to replace the part of the string that matched the pattern. Consider the following example:

```
$str = "It ain't going to rain no more, no more";
$str =~ s/ain't/is not/;
```

This changes `"ain't"` to `"is not"` in `$str`

The substitute operator can have two modifiers, `g` and `i`. The `g` modifier tells the substitute operator to find all matches in the given string and replace all of them:

```
$str = "Rob, Robbie, and Robette were siblings";
$str =~ s/Rob/Bob/g;
```

This changes `$str` to `"Bob, Bobbie, and Bobette were siblings"`.

The `i` modifier, which tells the pattern matcher to ignore the case of letters, can also be used with the substitute operator, as shown in the following code:

```
$str = "Is it Rose, rose, or ROSE?";
$str =~ s/Rose/rose/ig;
```

This changes `$str` to `"Is it rose, rose, or rose?"`.

All of the other details of Perl regular expressions can be found by typing the following:

```
perldoc perlre
```

8.9.4 The Transliterate Operator

Perl has a transliterate operator, `tr`, which translates a character or character class to another character or character class, respectively. For example, the following statement replaces all semicolons in `$str` with colons:

```
$str =~ tr/;/:/;
```

This particular operation can also be done with the substitute operator, as follows:

```
$str =~ s/;/:/g;
```

The following statement transforms all uppercase letters in `$str` to lowercase letters:

```
$str =~ tr/A-Z/a-z/;
```

Specific characters can be deleted from a string by using a null substitution character. For example, this next statement deletes all commas and periods from the string in `$str`:

```
$str =~ tr/\,\.//;
```

The transliterate operator also can be used on `$_` by omitting the left side and the binding operator.

8.10 File Input and Output

Files are referenced through program variables called *filehandles*, whose names do not begin with special characters. To make them more readable, filehandles are often spelled using all uppercase letters. Filehandles are initialized with the `open` function, which opens a file for use and assigns the filehandle. The `open` function establishes the relationship between a filehandle and the file's external (operating system) name. Files can be opened for input or either of two kinds of output. The first parameter to `open` is the filehandle; the second specifies both the file's external name and how it will be used. The file's name is either a literal string or a string-valued expression. The file's usage is specified by appending one or two special characters to the beginning of the file's name. The most common of these are shown in Table 8.5.

Table 8.5 File use specifications

Character(s)	Meaning
<	Input (the default)
>	Output, starting at the beginning of the file
>>	Output, starting at the end of the existing data on the file
+>	Input from and output to the file

The > file use specification indicates the file is to be opened for output and writing is to begin at the file's beginning (overwriting any data currently written there). If the file does not exist, it is implicitly created. >> also indicates the file is to be opened for output, but the new data to be appended to the end of the file's current data. The +< file use specification is used when a file's data is to be read, processed, and rewritten to the same file, replacing the data that was read. As with <, if the file does not exist, it is an error.

Every file has an internal file pointer that points to the position in the file where the next read or write will take place. The > and < file use specifications initialize the file pointer to the beginning of the file. The >> file use specification initializes the file pointer to the end of the current data in the file.

Because `open` can fail, it is often used with the `die` function, as shown in the following example:

```
open(INDAT, "<temperatures") or
     die "Error – unable to open temperatures $!";
```

This use of `die` after the `or` operator works for two reasons: First, `or` has lower precedence than the call to `open`. Second, `open` returns false if it fails.

Files are closed with the `close` function.

One line of text can be written to a file with the `print` function, using the file's filehandle as the first parameter, as follows:

```
print OUTDAT "The result is: $result \n";
```

Notice that no comma appears between the filehandle and the string.

Lines of text can be read from a file using the line input operator, including the filehandle between the angle brackets, as shown in the following example:

```
$next_line = <INDAT>;
```

Multiple lines can be read from a file with the `read` function. Because of the overhead of starting a file read or write operation, large files are input most quickly by reading more than one line at a time. The general form of a call to `read` is as follows:

`read(`*filehandle*`,` *buffer*`,` *length* `[,` *offset*`]);`

The *offset* is optional, as indicated by the brackets. The *buffer* is a scalar variable into which the lines that have been read are placed. The *length* parameter is the number of bytes of input to be read. When included, the offset is the distance from the beginning of the buffer where the input is to go. When not included in the call to `read`, the offset is zero. The `read` function returns the number of bytes that it read and placed in the buffer. Newlines count in the `read` operation.

Suppose you have a file whose filehandle is `ANIMALS`, which has lines of 50 characters, not counting the newline. You could read five lines from this file with the following statement:

```
$chars = read(ANIMALS, $buf, 255);
```

If `$chars` is less than `255` after this statement is executed, there were not 255 characters left in the file.

The lines in the buffer can be separated into separate elements of an array with the following statement:

```
@lines = split /\n/, $buf;
```

Some applications read a file, modify the data read from the file, and then rewrite the file with the modified data. The +> file use specification allows a file to be both read and written, but after the file has been read, its file pointer is left at the end of the data that has been read. To rewrite the file, its file pointer must be moved back to the file's beginning. This can be done with the `seek` function, which takes three parameters: the filehandle of the file, an offset, and a base position in the file. The possible base position values are 0, 1, or 2, specifying the beginning of the file, the current position of the file pointer in the file, or the end of the file, respectively. The offset is used to specify the number of bytes from the given base position. A positive value is an offset toward the end of the file (from the base position); a negative value is an offset in the direction of the beginning of the file. Most commonly, `seek` is used to rewind a file,[9] which sets the file pointer to the beginning of the file. This is specified with the following:

```
seek(filehandle, 0, 0);
```

When files are used to store data for CGI programs, they frequently need to be protected against corruption caused by multiple simultaneous writes. This is done with file locks, which are discussed and exemplified in Chapter 9.

8.11 An Example

The next sample program illustrates some of the features of Perl described in this chapter. The program reads employee records from a file and computes some statistics on the contents of the file. The precise specification for the program is included at its beginning as documentation.

```
# wages.pl - An example program to illustrate some of the
#            features of Perl
#  Input: A file of lines of employee data, where each line has
#         name:age:department code:salary
# Output: 1. The names of all employees whose names end with "son"
#         2. Percentage of employees under 40 years old
#         3. Average salary of employees under 40 years old
#         4. An alphabetical list of employees who are under 40
#            years old and who have salaries more than $40,000
```

9. Rewind is a holdover word from the days of magnetic tape, when setting the file pointer to the beginning required the tape to be rewound.

```
#>>> Open the data file and display a header for employees
#>>>  whose names end in 'son'
open(EMPLOYEES, "employees.txt") || die "Can't open employees $!";
print "Names that end in 'son'\n\n";

#>>> Loop to read and process the employee data
while (<EMPLOYEES>) {

#>>> Increment the number of employees and chop off the newline
    $total_employees++;
    chomp;

#>>> Split the input line into its four parts
    ($name, $age, $dept, $salary) = split(/:/);

#>>> If the name ends in 'son', print the name
    if ($name =~ /son$/) {
        print "$name \n";
    }

#>>> If the employee is under 40, count him or her and add his or
#>>>  her salary to the sum of such salaries
    if ($age < 40) {
        $under_40++;
        $salary_sum += $salary;

#>>> If the salary was over 40,000, add the person and his or her
#>>>  salary to the hash of such people
        if ($salary > 40000) {
            $sublist{$name} = $salary;
        }
    }
}
#>>> If there was at least one employee, continue
if ($total_employees > 0) {

#>>> If there was at least one under 40, continue
    if ($under_40 > 0) {

#>>> Compute and display the % of employees under 40 and their
#>>>  average salaries
        $percent = 100 * $under_40 / $total_employees;
        print "\nPercent of employees under 40 is: $percent \n";
        $avg = $salary_sum / $under_40;
        print "Average salary of employees under 40 is: $avg \n";
```

```
#>>> If there was at least one under 40 who earned a
#>>>  salary > 40,000, continue
        if (keys(%sublist)) {

#>>> Sort and display the names of the employees under 40 with
#>>>  with salaries > 40,000
            print "Sorted list of employees under 40",
                  " with salaries > \$40,000 \n";
            @sorted_names = sort (keys(%sublist));
            print "\nName \t\t Salary\n";
            foreach $name (@sorted_names) {
                print "$name \t \$$sublist{$name} \n";
            }
        }
        else {
            print "There were no employees under 40 who earned";
            print "over $40,000 \n";
        }  #>>> of if (keys(%sublist))
    }
    else {
        print "There were no employees under 40 \n";
    }  #>>> of if ($under_40 > 0)
}
else {
    print "There were no employees\n";
}  #>>> of if ($total_employees > 0)
```

Summary

Perl began as a relatively small language that included the capabilities of `awk` and `sh`, but it since has evolved to a full-blown programming language.

Perl's scalar values can be numbers, strings, or references. Numeric literals can be in either integer or floating-point form. Strings can be delimited by either single or double quotes. Variables that appear in double-quoted strings are interpolated into the string, but in single-quoted strings they are not. The names of all scalar variables begin with dollar signs. Numeric expressions can use the usual complement of arithmetic operators.

There are only two string operators: catenation (`.`) and repetition (`x`). However, there is a large collection of functions and operators that perform the most commonly needed operations on strings. Assignment statement operators can be simple or compound, where the compound operators combine a binary operator with the assignment operator. Keyboard input is obtained with the line input operator using `STDIN`. Screen output is created with the `print` function.

Perl includes two sets of relational operators, one for numeric operands and one for string operands. If an operand of the wrong type appears on one of these, it is coerced to the proper type. Perl also includes two sets of Boolean operators, the only difference being the level of precedence of the two (`&&`, `||`, and `!` have higher precedence; `and`, `or`, and `not` have lower precedence). In all of the control constructs, the statements whose execution is to be controlled are specified by a block. The two selection statements are `if` and its complement `unless`. The two logically controlled loops are `while` and `until`. The `for` statement can also be used as a logically controlled loop but is usually used for counting loops. Perl has no `case` or `switch` statement.

An array is a variable that stores lists. A list literal is a parenthesized list of scalar expressions or values. The names of all arrays begin with at signs (`@`). Individual elements of arrays are referenced through subscripts, which are numeric-valued expressions delimited by brackets. Arrays, like scalars, do not need to be declared—that is, they are implicitly declared. The length of an array is determined by the highest subscript with which it has been assigned a value. The `foreach` statement provides a convenient way of processing all of the elements of an array. The `shift`, `unshift`, `pop`, and `push` operators provide simple ways of adding or removing an end element of an array.

A hash is an associative array, which is a data structure in which each element is actually a key/value pair. The values are the stored data; the keys are used to find specific data values. There are no hash literals, so list literals with alternating keys and values are used. A particular value element of a hash is referenced by the hash's name followed by the key's name, delimited by braces. Hash elements can be removed with the `delete` operator.

A reference variable stores the address of a variable or a literal. References to variables are created with the backslash operator appended to the beginning of the variable's name. A reference to a list literal is created by placing the list literal in brackets. A reference to a hash literal is created by placing the list of hash elements in braces. A reference can be dereferenced by appending a dollar sign to the beginning of its name. In the cases of arrays and hashes, the `->` operator can also be used for dereferencing.

A function consists of the function header, which is the reserved word `sub` and the function's name, and a block that defines its actions. Local variables in functions are created with the `my` declaration. Function parameters are passed through the implicit array `@_`. To achieve pass-by-reference semantics, `@_` can be directly manipulated in the function. To achieve pass-by-value semantics, the values in `@_` are assigned to new local variables at the beginning of the function. Pass-by-reference semantics can also be achieved by passing references.

Perl has a powerful pattern matcher. Typically, the pattern is delimited by slashes and is matched against `$_`. The result of a pattern match is either `true` or `false`. String matches of subpatterns can be remembered in predefined variables by parenthesizing those subpatterns. The substitution operator is a powerful tool for modifying text. The transliteration operator is used to do literal translations of either characters or character classes.

Files are referenced through program variables called filehandles, which do not begin with any special characters. Files can be opened for input, output to the current end of the file, output to the beginning of the file, or input and output. These are specified with the `open` function, whose parameters are the filehandle and the file's external name as a string literal. Using a filehandle as the first parameter to `print` causes its output to go to that file. A single line from a file can be read using the line input operator with the filehandle. The `read` function can be used to read multiple lines from a file.

Review Questions

8.1 What are the three categories of Perl variables?

8.2 How many numeric data types does Perl have?

8.3 What is the purpose of the `qq` operator?

8.4 In what two ways do single-quoted string literals differ from double-quoted string literals?

8.5 What is the numeric value of `undef`?

8.6 Describe the operands and the actions of the `x` string operator.

8.7 Describe the parameters and actions of the `chomp` function.

8.8 Describe the parameters and actions of the `join` function.

8.9 What is a filehandle?

8.10 Under what conditions is a string used in a Boolean context considered to be true?

8.11 Why does Perl have two sets of relational operators?

8.12 What is the difference between Perl's two sets of Boolean operators?

8.13 What exactly does the expression `<STDIN>` do when executed?

8.14 In what three fundamental ways do Perl arrays differ from the arrays of other common high-level programming languages?

8.15 If an array's name appears in scalar context, what is its value?

8.16 What two predefined Perl functions can be used to implement a queue in an array?

8.17 What are the two fundamental ways in which hashes differ from arrays?

8.18 What statement adds the element `(joe, 42)` to the hash `%guys`?

8.19 How do you get the address of the scalar variable `$fruit`?

8.20 How do you get the address of a list literal?

8.21 What is a function declaration?

8.22 How do you create local variables in a function?

8.23 What are actual parameters? What are formal parameters?

8.24 What are the two ways a value can be returned from a function?

8.25 How does the `i` modifier change pattern matching?

8.26 Describe the two parameters for the substitute operator.

8.27 Describe the transliterate operator.

8.28 Describe the four file use specifications.

8.29 Under what circumstances is the `read` function used?

8.30 What is a file pointer?

8.31 What is one common use of the `seek` function?

Exercises

Write, test, and debug (if necessary) Perl programs for the following specifications.

8.1 *Input*: Three numbers, `a`, `b`, and `c`, each on its own line, from the keyboard

Output: The value of the expression `10ab-((c-1)/17.44)`

8.2 *Input*: A text file, specified on the command line, in which each line contains one number

Output: The second smallest number in the file, along with its position in the file, with 1 being the position of the first number

8.3 *Input*: Three names, on separate lines, from the keyboard

Output: The input names in alphabetical order, without using arrays

8.4 *Input*: A file of lines of text, specified on the command line

Output: Every input line that has more than 10 characters (not counting the newline) but fewer than 20 characters (not counting the newline) that contains the string `"ed"`

8.5 *Input*: A list of numbers in a file specified on the command line

Output: Two lists of numbers, one with input numbers that are greater than zero, and one with those that are less than zero (ignore the zero-valued numbers)

Method: You must first build two arrays with the required output numbers before you display any of them.

8.6 *Input*: A file that contains English words, where each word is separated from the next word on a line by one space, specified on the command line

Output: A table, in which the first column has the unique words from the input file and the second column has the number of times the word appeared in the file; no word can appear twice in the table

Method: Your program must use two arrays to store the table, one for the words and one for the frequency values.

8.7 *Input*: A file in which each line contains a string of the form `name+sales`, where in some cases the sales will be absent (but not the plus sign), specified on the command line

Output: A list of the names and sales numbers that remain after the following processing:

a. Names with sales numbers are added to a hash when they are first found, along with their sales numbers
b. Names with absent sales numbers are deleted from the hash if they are already there
c. When a name appears that is already in the hash, the new sales number is added to the old sales number (the one already in the hash)

8.8 *Input*: A file specified on the command line that contains text

Output: The input text after the following modifications have been made:

a. All multiple spaces are reduced to single spaces
b. All occurrences of `Darcy` are replaced with `Darcie`
c. All lines that begin with ~ are deleted
d. All occurrences of `1998` are replaced with `1999`

8.9 *Input*: A file specified on the command line that contains a C program

Output: For each line of the input:

a. The number of words (variables and reserved words) on the line
b. The number of numeric literals without decimal points on the line
c. The number of numeric literals with decimal points on the line
d. The number of braces and parentheses on the line

Write functions for the specifications in Exercises 8.10, 8.11, and 8.12

8.10 *Parameter*: An array of strings, passed by value

Return value: A list of the unique strings in the parameter array

8.11 *Parameter*: An array of numbers

Return value: The average and median of the parameter array

8.12 *Parameter*: A reference to an array of strings

Return value: A list of the unique strings in the parameter array

CHAPTER

9

Using Perl for CGI Programming

9.1 The Common Gateway Interface
9.2 CGI Linkage
9.3 Query String Format
9.4 The `CGI.pm` Module
9.5 A Survey Example
9.6 Cookies
Summary • Review Questions • Exercises

This chapter introduces the Common Gateway Interface (CGI) and the use of Perl for CGI programming. It begins with an overview of CGI, explaining how CGI programs are linked to Web documents and how results are returned to clients from CGI programs. Next, the format of query strings is described. Then it discusses the `CGI.pm` module, which provides a simple and efficient way to write CGI programs in Perl. A CGI program to process a product order form is then presented. This is followed by another complete example, including the XHTML code to describe a survey form and the two CGI programs that are used to provide the processing required for the form. The chapter closes with an introduction to cookies and how they are created and used in Perl CGI programs.

9.1 The Common Gateway Interface

XHTML is a markup language and, as such, cannot by itself describe computations, allow interaction with the user, or provide access to a database. Yet these are things that are commonly needed in Web-based systems. Computations are required to provide support for all kinds of Web commerce. The Web also is now a common way to access databases—for example, for making reservations for transportation services. Such applications obviously require interaction with clients. One early response to these needs was to develop a technique for a browser to access the software resources of the server machine. Using this approach, a browser can run programs indirectly on the server, including database access systems for the databases that reside on the server. These server-based programs communicate back to the client through HTTP. The protocol that is used between a browser and software on the server is called the *Common Gateway Interface*.

Before discussing CGI further, it is useful to take a brief look at its alternatives. There are now several popular approaches to providing server-side computation, including support for dynamic documents. As just discussed, the first of these uses the CGI to run software on the server that is completely separate from markup documents. CGI is part of a general approach that is often called LAMP, which is an acronym for Linux, Apache, and MySQL. The 'P' part of the acronym is generic—it represents one of the server-side programming and/or scripting languages, Perl, PHP, or Python. This chapter discusses using Perl and CGI. Chapter 11, "Introduction to PHP," discusses using PHP as an XHTML-embedded, server-side scripting language. This book does not cover Python, another scripting language. MySQL, a database management system, as used in conjunction with Perl, Java, and PHP in Web database access systems, is discussed in Chapter 13, "Database Access through the Web."

A second approach to providing server-side computation and dynamic documents is Microsoft's product, ASP. The latest incarnation of ASP is ASP.NET, an important part of the .NET computing platform. ASP.NET documents are mixtures of XHTML markup and either programming language code or references to external programs. These documents are compiled into classes in which the XHTML code is replaced by output statements that produce that XHTML code. ASP.NET documents can use any of the .NET programming languages. ASP.NET is discussed in Chapter 12, "Introduction to ASP.NET."

Yet another approach to providing server-side computation and dynamic documents is Java servlets and Java Server Pages (JSP). Servlets are Java classes with close connections to XHTML documents, which allow them to interact with Web clients conveniently. JSP is similar to ASP.NET in that programming code can be embedded in markup documents and programs can be referenced in documents to provide computational support. The primary difference between JSP and ASP.NET is that JSP restricts the developer to Java rather than a range of programming and scripting languages. JSP documents are converted to Java servlets before they are compiled. Both servlets and JSP are discussed in Chapter 10, "Servlets and Java Server Pages."

Now we can return to CGI. An HTTP request to run a CGI program is like any other HTTP request except that the requested file can be identified by the server as being a CGI program. Servers can identify CGI programs by their addresses on the server or by their filename extensions. When a server receives a request for a CGI program, it does not return the file—it executes the program in the file and returns that program's output.

A CGI program can produce results in a number of different forms. Most often, an XHTML document is returned. When a new document is generated, it consists of two parts: an HTTP header and a body. The header must be complete if the response is going directly to the client. It can be partial if the response is going to the client through the server, which completes the header. This is the usual process. The form of the body, which is specified in the header, can be XHTML, plain text, or even an image or audio file.

One common way for a browser user to interact with a Web server is through forms. A form is presented to the user, who is invited to fill in the text boxes and click the buttons of the form. The user submits the form to the server by clicking its *Submit* button. The contents of the form are encoded and transmitted to the server. The server must use a program to decode the transmitted form contents, perform whatever computation is necessary on the form data, and produce its output. Figure 9.1 shows the communications configuration for CGI.

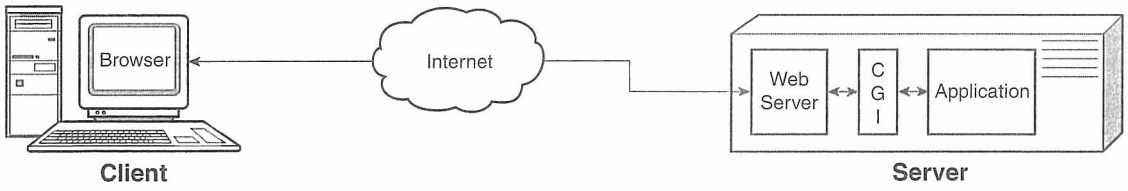


Figure 9.1 Communications and computation using CGI

Both CGI programs and client-side scripts, such as JavaScript, support dynamic documents. However, client-side scripts cannot access files and databases on the server.

9.2 CGI Linkage

This section describes how the connection between an XHTML document being displayed by a browser and a CGI program on the server is established.

CGI programs often reside in one specific directory on the server, `cgi-bin`, which is usually a subdirectory of `public_html`[1]. However, the Web server administrator can choose a different directory name and place for CGI

1. `public_html` is the directory in which the publicly accessible XHTML document files are stored.

programs. The only requirement is that the CGI programmer knows the location of CGI programs. The examples in this chapter assume that CGI programs are stored in a directory named `cgi-bin`, which is a subdirectory of the directory in which XHTML documents are stored.

A server can be configured to recognize the requested file as being a program by the extension on the file's name. For example, if the extension on the requested file's name is `.pl`, that could indicate that it is a Perl CGI program. Another extension that indicates the file is a Perl program is `.cgi`. Because CGI programs are invoked by the server, their access protection code must be set to allow this.

If a CGI program has been compiled and is in machine code, the server can invoke it directly. However, the compiled versions of Perl programs normally are not saved and are not in machine code anyway, so the `perl` system must be invoked on every Perl CGI program.[2] For UNIX-based Web servers, this can be done by adding a special line to the beginning of the program that specifies that `perl` must be run, using the remainder of the program file as input data to `perl`. This special line also must specify the location of `perl`. On UNIX systems, `perl` is often stored in `/usr/local/bin`. If so, the first line is as follows:

```
#!/usr/local/bin/perl -w
```

The `#!` specifies that the program whose location follows must be executed on the rest of the file. And, of course, `/usr/local/bin/perl` is the path to the `perl` system. If `perl` resides in some other directory, that directory's name is used in place of `/usr/local/bin`. Because `!` is called "bang" in the UNIX world, the `#!` line is often called "shebang."

Windows-based Web servers do not require the shebang line in Perl CGI programs.

An XHTML document can specify a call to a CGI program using an anchor tag (`<a>`), which must include a hypertext reference attribute (`href`). In this case, a CGI program call is similar to a link to an XHTML document. The value of `href` must be the complete URL of the CGI program's filename. Consider the following anchor tag:

```
<a href = "./cgi-bin/reply.cgi">
Click here to run the CGI program reply.cgi
</a>
```

The content of the anchor tag specifies the actual link in the XHTML. In this example, `cgi-bin`, which is a subdirectory of where the calling XHTML document is stored, is the directory where CGI programs are stored, and `reply.cgi` is the name of the CGI program. Some educational institutions now use a system called CGI Wrap to allow students to run CGI programs in a relatively secure environment This system allows the CGI program to be in the

2. Systems are available that produce and save compiled versions of Perl programs for use on Web servers. For example, look at `http://perl.apache.org/list/mod_perl.html`.

student's directory rather than on the server system. For CGI Wrap, Perl CGI programs must use the `.cgi` extension to their filenames.

The most common application of CGI programs is form processing, so most CGI programs are not called from XHTML documents with anchor tags. Rather, they are called as a side effect of *Submit* button clicks, with the CGI program being specified on the action attribute of the form tag. This is exemplified in Section 9.4.2.

The following document calls a trivial Perl CGI program through its anchor tag:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- reply.html
     A trivial document to call a simple Perl CGI program
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> XHTML to call the Perl CGI program, reply.cgi
    </title>
  </head>
  <body>
    <p>
      This is our first Perl CGI example
      <br /><br />
      <a href = "./cgi-bin/reply.cgi">
      Click here to run the CGI program, reply.cgi
      </a>
    </p>
  </body>
</html>
```

We have implied that the CGI program is always on the server that sent the XHTML document to the browser. However, the `href` attribute can be assigned the URL of any CGI program on any Web server on the Internet.

The obvious next step is to describe the CGI program, `reply.cgi`, which simply returns a greeting to the browser (through the server). The connection from a CGI program to the client is through standard output. Therefore, anything the CGI program sends to standard output is sent to the server, which sends it on to the client. In a Perl CGI program, this means that the `print` function is used to communicate to the client.

As previously stated, the HTTP response to the client can be in a variety of different forms, but it must begin with the HTTP header. The CGI program

often supplies only the first line of the header; the remainder is added by the server before the response document is sent to the client. The first line of the header specifies the form of the response as a MIME content type. In this book, we consider only XHTML as the response content type, which is specified with `text/html`.

The line following the one-line header *must* always be blank. The blank line indicates the end of the HTTP header. Therefore, the first output from all Perl CGI programs that return partial headers and XHTML response bodies is generated with the following statement:

```
print "Content-type: text/html \n\n";
```

Since the output of a CGI program is ultimately for the browser, it must be in the form of an HTML (or XHTML) document. Such a document is created in Perl by using `print` statements to output the XHTML text.

If the only task of a CGI program is to send one line of information, most of the program will be used to generate the required tags for the XHTML file. The following is the Perl program `reply.cgi`:

```
#!/usr/local/bin/perl -w
# reply.cgi
# This CGI program returns a greeting to the client

print "Content-type: text/html \n\n",
 "<?xml version = '1.0' encoding = 'utf-8'?> \n",
 "<!DOCTYPE html PUBLIC '-//w3c//DTD XHTML 1.1//EN'\n",
 "'http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd'>\n",
 "<html xmlns = 'http://www.w3.org/1999/xhtml'>\n",
 "<head><title> reply.cgi example </title></head>\n",
 "<body>\n",
 "<h1> Greetings from your Web server! </h1>\n",
 "</body></html>";
```

9.3 Query String Format

HTTP requests made by the GET and POST methods transmit the data associated with the request to the Web server. This data is in the form of a character string called a *query string*. This section describes the format of a query string.

Query strings can be handwritten and used directly with GET and/or POST HTTP methods. However, the great majority of query strings are those that code form data and are constructed by the browser when the *Submit* button of the form is clicked. The following paragraphs describe query strings in terms of form data.

For each element (widget) in the form that has a value, the widget's name and that value are coded as a character string of the form *name=value* and

included in the query string. A widget's value is always a character string. For example, if the value of a radio button named `payment` is `discover`, it is coded in the query string as follows:

```
payment=discover
```

If the form has more than one widget, their name/value pairs are separated by ampersands (`&`):

```
caramel=7&payment=discover
```

If there are special characters in the value of a widget, they are coded as a percent sign (`%`) followed by a two-character hexadecimal number that is the ASCII code for that character.[3] For example, the hexadecimal ASCII code for the exclamation point (`!`) is `21`, so that character is coded as `%21`. Likewise, because the hexadecimal ASCII code for a space is `20`, spaces are coded as `%20`. Suppose we had a form whose only widgets were a radio button collection named `payment` whose value was `visa` and a text widget whose name was `saying` and whose value was `"Eat your fruit!"`. The query string for this form would be as follows:

```
payment=visa&saying=Eat%20your%20fruit%21
```

Some browsers replace spaces in form data values with plus signs (+). For these, the previous example would be coded as follows:

```
payment=visa&saying=Eat+your+fruit%21
```

Recall from Chapter 2, "Introduction to XHTML," that the `method` attribute of `<form>` specifies one of the two techniques, `get` or `post`, used to pass the form data to the server. `get` is the default, so if no `method` attribute is given in the `<form>` tag, `get` will be used. The alternative technique is `post`. In both techniques, the form data is coded into a query string when the user clicks the *Submit* button. When the `get` method is used, the browser attaches the query string to the URL of the CGI program, so the form data is transmitted to the server with the URL. The browser inserts a question mark at the end of the actual URL just before the first character of the query string so that the server can easily find the beginning of the query string. The server removes the query string from the URL and places it in the environment variable `QUERY_STRING`, where it can be accessed by the CGI program. The `get` method can also be used to pass parameters to the server when forms are not involved (this cannot be done with `post`). The main disadvantage of the `get` method is that some servers place a limit on the length of the URL string and truncate any characters past the limit. So, if the form has more than a few widgets, `get` is not a good choice.

The following is an example of a URL with a query string:

```
http://cs.ucp.edu/cgi-bin/rws/test.cgi?sender=bob&day=2
```

3. When the `GET` method is used, the query string becomes part of a URL, in which certain special characters can cause problems.

When the post method is used, the query string is passed through standard input to the CGI program so that the CGI program can simply read the string. The length of the query string is passed through the environment variable CONTENT_LENGTH. There is no length limitation for the query string with the post method, so it is obviously the better choice when there are more than a few widgets in the form.

The following code, as part of a CGI program that handles GET or POST requests, fetches the query string and places it is $query_string, regardless of which method was used.

```
$request_method = $ENV{'REQUEST_METHOD'};
if ($request_method eq "GET") {
   $query_string = $ENV{'QUERY_STRING'};
}
elsif ($request_method eq "POST") {
   read(STDIN, $query_string, $ENV{'CONTENT_LENGTH'})
}
else {
   print "Error - the request method is illegal \n";
   exit(1);
}
```

Notice that although the value of the method attribute of the form element in XHTML must be lowercase, when that value is stored in the REQUEST_METHOD environment variable, it is in uppercase.

9.4 The CGI.pm Module

Much of what a CGI program must do is routine—that is, it is nearly the same for all CGI programs. Among these common tasks are creating the required tags, such as <html> and </html>, and creating form elements such as buttons and lists. Therefore, it is natural to have standard routines to do these things. CGI.pm, which was developed by Lincoln Stein, is a Perl module of functions for these common tasks.[4] The CGI.pm module is part of the standard Perl distribution. Its documentation can be obtained with the following command:

```
perldoc CGI
```

A Perl program specifies that it needs access to a particular module with the use declaration. In the case of CGI.pm, only a part of the module is usually needed. The part we need here, which is the most often used part, is named :standard. The following declaration provides access to this collection of resources from CGI.pm:

```
use CGI ":standard";
```

4. A Perl module can be thought of as a library of Perl functions.

9.4.1 Common CGI.pm Functions

Many of the functions in CGI.pm produce XHTML tags. In these cases, the functions have the names of their associated XHTML tags. These functions are called *shortcuts*. Shortcut functions produce their tags by using the parameters passed to them for the attributes and content of the tag. They may also generate embedded tags. If there are no attributes and no embedded tags, the only parameter is the content of the tag. A shortcut function that takes no parameters may be called without the parentheses. For example, the following call to br:

```
br;
```

returns

```
<br/>
```

Note that the shortcut functions do not produce any output—they simply return strings. So, to get a break in the output, use the following:

```
print br;
```

A shortcut function may take several different kinds of parameters, including a scalar literal or variable, a reference to a hash, or a reference to an array or list. When there is just one parameter, the CGI function call has the same form as that for any other Perl function. For example, in the following call, the only parameter is the content of the tag:

```
print h1("This is the real stuff");
```

This produces the following:

```
<h1> This is the real stuff </h1>
```

CGI programs sometimes create and send forms back to the client to gather information. In the following paragraphs, we use some examples of CGI.pm functions that produce widgets. Tags can have both content and attributes. In the previous example of the h1 function, the parameter becomes the content of <h1>. Each attribute of a tag is passed in the name/value pair form that is used in a hash literal, where the name and value are separated by =>. The name of the attribute is the key in the pair, and the attribute value is its value. The attribute names are preceded by minus signs. For example, consider the following call to textarea:

```
print textarea(-name => "Description",
               -rows => "2",
               -cols => "35"
);
```

This produces the following:

```
<textarea name="Description" rows="2" cols="35">
</textarea>
```

If both attributes and content are passed to a shortcut function, the attributes are specified in a hash literal, which must be the first parameter. For example, the statement

```
print a({-href => "fruit.html"},
      Press here for fruit descriptions");
```

generates

```
<a href="fruit.html"> Press here for fruit descriptions </a>
```

One convenient characteristic of the shortcut functions is that the tags and their attributes are distributed over the parameters to the function. For example, consider the following call:

```
print ol(li({-type => "square"},
      ["milk", "bread", "cheese"]));
```

Notice that a reference to the literal array, `("milk", "bread", "cheese")`, is passed as a parameter to `li`. In this example, the `li` tag and its attribute are distributed over the list items. This produces the following:[5]

```
<ol>
    <li type="square" milk</li>
    <li type="square" bread</li>
    <li type="square" cheese</li>
</ol>
```

As another example, consider the following call to `radio_group`, in which the collection name is specified just once, and the values can be specified as a reference to an array:

```
print radio_group(-name => 'colors',
              -value => ['blue', 'green', 'yellow', 'red'],
              -default => 'blue');
```

This produces the following:

```
<input type="radio" name="colors" value="blue" checked /> blue
<input type="radio" name="colors" value="green" /> green
<input type="radio" name="colors" value="yellow" /> yellow
<input type="radio" name="colors" value="red" /> red
```

The `CGI.pm` functions that are not shortcuts often produce the boilerplate that is part of every XHTML document. The first thing most CGI programs must do is produce the content type line. The `header` function does precisely this. Specifically,

```
print header;
```

5. Actually, the produced XHTML is returned as a single string.

creates

```
Content-type: text/html;charset=ISO-8859-1
-- blank line --
```

Notice that `CGI.pm` includes the `charset` value `ISO-8859-1` in the output of `header`. We do not include `charset` in the HTTP headers of our non-`CGI.pm` examples because the default character set, ISO-8859-1, is fine. If the content type is not `text/html`, the alternative content type can be specified as the parameter to `header`.

The next required XHTML output of a CGI program is the head part of the document. This is generated by the `start_html` function, whose parameter specifies the title of the document. For example, the call

```
print start_html("Paul's Gardening Supplies");
```

produces

```
<!DOCTYPE html
PUBLIC "-//W3C//DTD XHTML 1.0 Transitional //EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" lang="en-US"
            xml:lang="en-US">
<head><title> Paul's Gardening Supplies </title>
<meta http-equiv="Content-Type" content="text/html;
                  charset=iso-8859-1"> </head>
<body>
```

Section 9.3 describes the format of the query string. Any form-processing program must first extract the form data from the query string. This means it must be split into its name/value pairs, which then must be split to access the values. Any characters coded as hexadecimal ASCII must be decoded, and any plus signs must be replaced by spaces. The whole process of decoding the query string and setting local variables to the values in the query string can be done by calls to the `CGI.pm` function, `param`. For example, if the query string is

```
name=Bob%20Brumbel&payment=visa
```

then the following statement sets `$name` to `"Bob Brumbel"`:

```
my $name = param("name")
```

So, for the whole query string, the following statement could be used:

```
my($name, $payment) = (param("name"), param("payment"));
```

This sets the local variables `$name` and `$payment` to the values of the form values for `"name"` and `"payment"` (`"Bob Brumbel"` and `"visa"`, respectively).

The last XHTML code required in a document is generated with `end_html`, which produces `</body> </html>`.

The `CGI.pm` functions for creating XHTML tables are discussed in Section 9.5.

9.4.2 A Complete Form Example

The following XHTML code describes a form for taking sales orders for popcorn. Three text widgets are used at the top of the form to collect the buyer's name and address. These are placed in a borderless table to force the text boxes to align vertically. A second table is used to collect the actual order. Each row of this table names a product with the content of a `<td>` tag, displays the price with another `<td>` tag, and uses a text widget with `size` set to `2` to collect the quantity ordered. The payment method is input by the user through one of four radio buttons. Recall that this form also appears in Chapter 2.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
  "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- popcorn.html
     This describes popcorn sales form page>
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Popcorn Sales Form </title>
  </head>
  <body>
    <h2> Welcome to Millenium Gynmastics Booster Club Popcorn
         Sales
    </h2>

<!-- The next line gives the address of the CGI program -->
    <form action = "./cgi-bin/popcorn.cgi"
          method = "post">
      <table>

<!-- Text widgets for name and address -->
        <tr>
          <td> Buyer's Name: </td>
          <td> <input type = "text" name = "name" size = "30">
          </td>
        </tr>
        <tr>
          <td> Street Address: </td>
          <td> <input type = "text" name = "street" size = "30">
          </td>
        </tr>
        <tr>
          <td> City, State, Zip: </td>
          <td> <input type = "text" name = "city" size = "30">
```

```
          </td>
        </tr>
      </table>
      <br />
      <table border = "border">

<!-- First, the column headings -->
        <tr>
          <th> Product Name </th>
          <th> Price </th>
          <th> Quantity </th>
        </tr>

<!-- Now, the table data entries -->
        <tr>
          <td> Unpopped Popcorn (1 lb.) </td>
          <td> $3.00 </td>
          <td> <input type = "text" name = "unpop" size ="2">
          </td>
        </tr>
        <tr>
          <td> Caramel Popcorn (2 lb. cannister) </td>
          <td> $3.50 </td>
          <td> <input type = "text" name = "caramel" size = "2">
          </td>
        </tr>
        <tr>
          <td> Caramel Nut Popcorn (2 lb. cannister) </td>
          <td> $4.50 </td>
          <td> <input type = "text" name = "caramelnut" size = "2">
          </td>
        </tr>
        <tr>
          <td> Toffey Nut Popcorn (2 lb. cannister) </td>
          <td> $5.00 </td>
          <td> <input type = "text" name = "toffeynut" size = "2">
          </td>
        </tr>

      </table>
      <br />

<!-- The radio buttons for the payment method -->
      <h3> Payment Method: </h3>
      <p>
```

```
        <label>
          <input type = "radio" name = "payment" value = "visa"
                 checked = "checked" /> Visa <br />
        </label>
        <label>
          <input type = "radio" name = "payment"
                 value = "mc" />  Master Card <br />
        </label>
        <label>
          <input type = "radio" name = "payment"
                 value = "discover"/> Discover <br />
        </label>
        <label>
          <input type = "radio" name = "payment"
                 value = "check" /> Check <br />
        </label>
      </p>

<!-- The submit and reset buttons -->
      <p>
        <input type = "submit" value = "Submit Order" />
        <input type = "reset" value = "Clear Order Form" />
      </p>
    </form>
  </body>
</html>
```

Figure 9.2 shows a browser display of `popcorn.html`. Figure 9.3 shows an example of this form after it has been filled out.

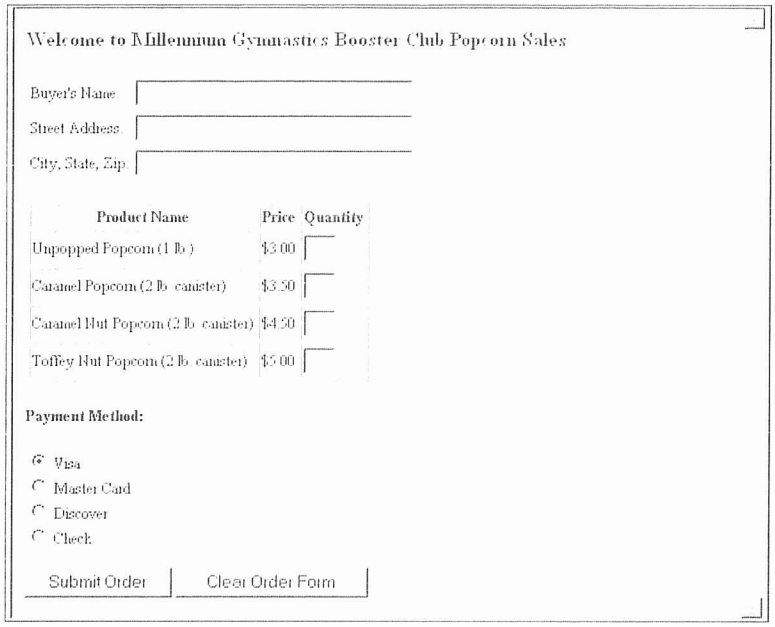


Figure 9.2 Display of `popcorn.html`

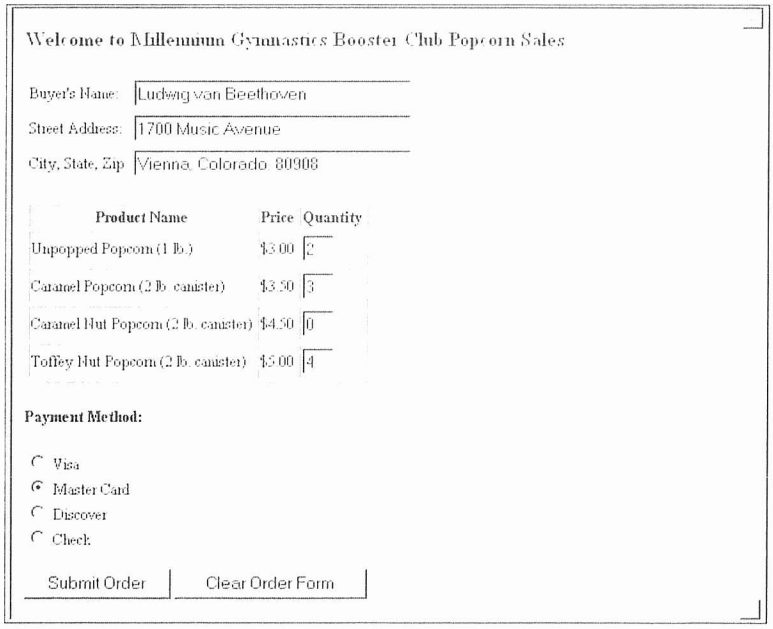


Figure 9.3 Display of `popcorn.html` after the form is filled out

We now can consider the CGI program that will process the data from the popcorn order form. The program must compute the costs of the ordered items, determine the total sale amount, and send them back to the user as an XHTML document. The program must determine what was ordered and compute its cost. For the item orders, the program must use the values to compute the total price and total number of items in the order. Finally, this program must build the XHTML code to reply to the user who placed the order. The complete program, called `popcorn.cgi`, follows:

```
#!/usr/local/bin/perl

# popcorn.cgi
# A CGI program to process the popcorn sales form

use CGI ":standard";

#>>> Initialize total price and total number of purchased items
$total_price = 0;
$total_itemi = 0;

#>>> Produce the header part of the HTML return value
print header;
print start_html("CGI-Perl Popcorn Sales Form, using CGI.pm");

#>>> Set local variables to the parameter values
my($name, $street, $city, $payment) =
      (param("name"), param("street"),
       param("city"), param("payment"));
my($unpop, $caramel, $caramelnut, $toffeynut) = (param("unpop"),
        param("caramel"), param("caramelnut"),
        param("toffeynut"));

#>>> Compute the number of items ordered and the total cost
$total_price = 3.0 * $unpop + 3.5 * $caramel + 4.5 * $caramelnut +
               5.0 * $toffeynut;
$total_items = $unpop + $caramel + $caramelnut + $toffeynut;


#>>> Produce the result information for the browser and
#>>>  finish the page
print h3("Customer:"), "\n";
print "$name<br/>\n", "$street <br/>\n", "$city <br/><br/>\n";
print "<b>Payment method:</b> $payment <br/><br/>\n";
print h3("Items ordered:"), "\n";
```

```
if ($unpop > 0) {print "Unpopped popcorn: $unpop <br/>\n";}
if ($caramel > 0) {print "Caramel popcorn: $caramel <br/>\n";}
if ($caramelnut > 0)
  {print "Caramel nut popcorn: $caramelnut <br/>\n";}
if ($toffeynut > 0)
  {print "Toffey nut popcorn: $toffeynut <br/>\n";}
print "Thank you for your order <br/><br/>\n";
print "<b>Your total bill is:</b> \$ $total_price <br/> \n";
print end_html;
```

Figure 9.4 shows the results of running `popcorn.cgi` on the filled out form shown in Figure 9.3.

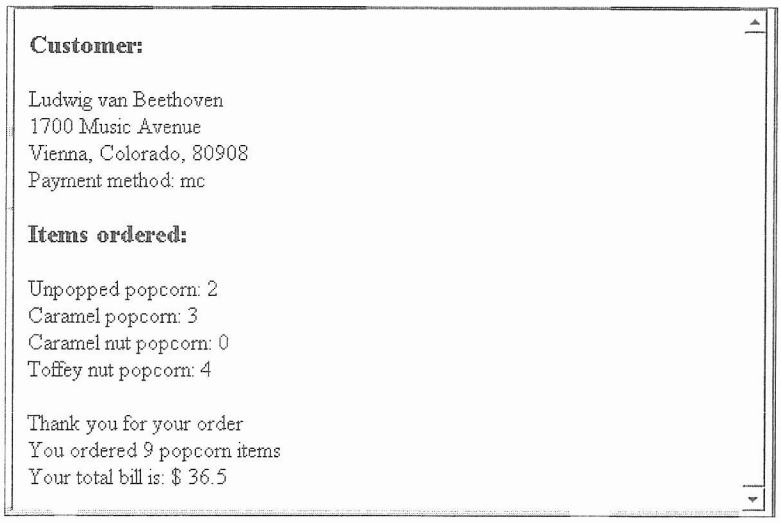


Figure 9.4 Output of `popcorn.cgi`

9.5 A Survey Example

It is common for a Web document to conduct a survey of the opinions or choices of the people who visit the document. The document to gather the information is similar to the popcorn sales example. The most important difference in the CGI program that processes the inputs for a survey is that the data must be stored between visits. The obvious place to store such data is on the disk on the server. This file can reside in the directory where CGI programs are stored. We use a file (rather than a database) in this example because of the simplicity of the data that needs to be stored. However, it many cases a database would be a better choice.

For this example, we use two CGI programs: one to process a client's response to the survey form and one to produce the latest totals from the survey, which will be produced when the user clicks a link in the document.

One of the complications with this application is that more than one user may submit survey forms concurrently, which could lead to corruption of the data if the changes occur at about the same time. For example, suppose two clients simultaneously request executions of a CGI program that reads a number from a file, increments the number, and rewrites the file. Consider the following scenario: The initial value in the file is 27. Two browsers simultaneously call the CGI program. The three steps of the CGI program (read the file, increment the value from the file, rewrite the file) can become interleaved. One possible sequence of these steps is as follows. (We name the two browser's executions of the CGI program CGI1 and CGI2.)

1. CGI1 reads the file into its variable, `counter` (value is 27).
2. CGI2 reads the file into its variable, `counter` (value is 27).
3. CGI1 increments its `counter` to 28.
4. CGI1 rewrites the file, which now has 28.
5. CGI2 increments its `counter` to 28.
6. CGI2 rewrites the file, which still has 28.

So, although the file now should have 29, it has 28.

Such multiple simultaneous accesses can be prevented by requiring the programs that access the file to lock the file before accessing it and unlock it when they are finished.[6] The Perl function `flock` can be used to both lock and unlock the file. `flock` takes two parameters: the filehandle of the file to be locked or unlocked, and the specified process (lock or unlock).

Actually, locking the file with `flock` does not directly block all accesses to the file; rather, it prevents any other process from acquiring the file's lock. So, it effectively blocks other processes from accessing the file only if they first attempt to lock the file. For our example we assume that no other process will attempt to use the file, so this works.

The values for the parameter to `flock` are defined in the Perl `Fcntl` module, which is made accessible with the following:

```
use Fcntl qw(:DEFAULT :flock);
```

Because closing a file implicitly unlocks it, we only need the parameter value for a write lock, which is `LOCK_EX`. The following example illustrates the process of updating a file using locking:

```
use Fcntl qw(:DEFAULT :flock);
open(TAX_DATA, "+<taxdata") or
   die "TAX_DATA could not be opened";
```

6. Locking the file means that all other accesses to it are temporarily blocked. Unlocking it makes the file available to other accesses.

```
flock(TAX_DATA, LOCK_EX) or
   die "TAX_DATA could not be locked";
chomp($tax = <TAX_DATA>);
# update $tax
# Rewind the file so the write goes to its beginning
seek(TAX_DATA, 0, 0) or
  die "TAX_DATA could not be rewound";
print TAX_DATA $tax or
  die "TAX_DATA could not be rewritten";
close TAX_DATA or die "TAX_DATA could not be closed";
```

The survey form collects votes on what consumer electronics device the client is most likely to purchase in the next six months. The form will also collect the age and gender of the client. A visitor to the document can vote, see the current voting results, or both. So the document needs two links, one to the CGI program that records a vote and one that produces a table of current results. The survey document is described by the following code:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- conelec.html
     A document to present the user with a consumer electronics
     purchasing survey form -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Consumer Electronics Purchasing Survey </title>
  </head>
  <body>
    <form action = "./cgi-bin/conelec1.cgi" method = "post">
      <h2> Welcome to the Consumer Electronics Purchasing Survey
      </h2>
      <p />
      <h4> Your Age Category: </h4>
      <p>
        <label>
          <input type = "radio" name = "age" value = "0"
                 checked = "checked" /> 10-25 <br/>
        </label>
        <label>
          <input type = "radio" name = "age" value = "1"/>
            26-40 <br/>
        </label>
        <label>
```

```
      <input type = "radio" name = "age" value = "2"/>
        41-60 <br/>
    </label>
    <label>
      <input type = "radio" name = "age" value = "3"/>
        Over 60 <br /> <br />
    </label>
  </p>
  <h4> Your Gender: </h4>
  <p>
    <label>
      <input type = "radio" name = "gender" value = "0"
             checked = "checked" /> Female <br/>
    </label>
    <label>
      <input type = "radio" name = "gender" value = "4"/>
        Male <br /> <br />
    </label>
  </p>
  <h4> Your Next Consumer Electronics Purchase will be: </h4>
  <p>
    <label>
      <input type = "radio" name = "vote" value = "0"/>
        Conventional TV <br />
    </label>
    <label>
      <input type = "radio" name = "vote" value = "1"/>
        HDTV <br />
    </label>
    <label>
      <input type = "radio" name = "vote" value = "2"/>
        VCR <br />
    </label>
    <label>
      <input type = "radio" name = "vote" value = "3"/>
        CD player <br />
    </label>
    <label>
      <input type = "radio" name = "vote" value = "4"/>
        Mini CD player/recorder <br />
    </label>
    <label>
      <input type = "radio" name = "vote" value = "5"/>
        DVD player <br/>
    </label>
```

```
        <label>
          <input type = "radio" name = "vote" value = "6"
                 checked = "checked" /> Other <br /> <br/>
        </label>

        <input type = "submit" value = "Submit Order" />
        <input type = "reset" value = "Clear Order Form"/>
      </p>
    </form>

    <hr/>
    <p>
      To see the results of the survey so far, click
      <a href = "./cgi-bin/conelec2.cgi"> here </a>
    </p>
  </body>
</html>
```

Figure 9.5 shows the display of `conelec.html`.

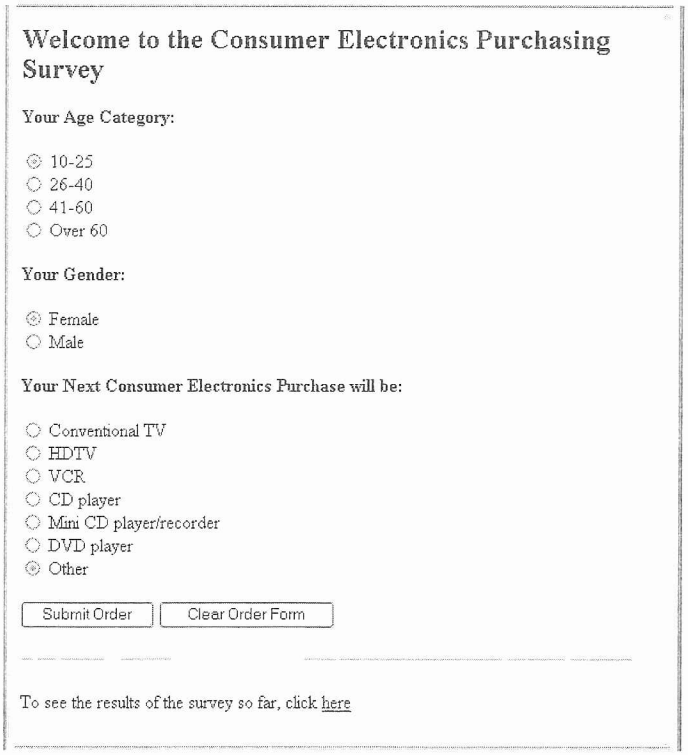


Figure 9.5 Display of `conelec.html`

Before we can develop either of the two CGI programs for this application, we must design the file format to be used to store the data. The simple format we will use is eight strings that represent the rows of voting totals. The first four rows store the votes from female visitors; the last four are for the male visitors. Initially, the file will have eight lines, each being a string of seven zeros (one for each possible vote), separated by spaces. The file must be created and initialized to all zeros before the survey page is made public. This can be done with a text editor because it is simple text.

Note in `conelec.html` that the values `0`, `1`, `2`, and `3` are used for the radio buttons for selecting the age group. Also, the values `0` and `4` are used for the gender selection. These values were chosen so that the index of the element of the vote data array that must be changed can be computed by adding the age group value and the gender value. For example, a male voter (gender = `4`) who selects the 41–60 age group (age = `2`) affects the element with the index `6`.

It is always a good idea to check that file operations succeeded. In Chapter 8, "The Basics of Perl," the `die` function is used with `open` to produce an error message and terminate the program when `open` fails. In a CGI program, this is a bad idea because if a file operation fails and the program dies, the client will not know what happened. Instead, a message must be returned to the client indicating the problem. In the two CGI programs for the survey form document, a function named `error` is included to produce a message and terminate the program. Because the message must go to the client, it must be in the form of XHTML.

The CGI program that collects and records the voting data is named `conelec1.cgi`. Its tasks are as follows:

1. Get the form values.
2. Determine which row of the file must be modified.
3. Open, lock, and read the survey data file.
4. Split the affected data string into numbers and store them in an array.
5. Modify the affected array element and join the array back into a string.
6. Rewind the data file (with `seek`).
7. Rewrite and close the survey data file.

We already know how to get the widget values from the query string, and reading and writing the survey data file is relatively simple. Its lines will be read into an array of strings, one array element per row of the file. Each of the strings stores seven numbers. Because each submitted voting form changes just one element of the file, only one row must be converted from a string to an array of numbers. As previously explained, the row to be changed can be determined by adding the `age` and `gender` form data values. The `gender` form value determines which half of the file is affected. The `age` form data will determine which of the four rows of the `gender`'s half is affected. The actual change amounts to splitting the row into an array of numbers, adding 1 to one of its elements,

which is determined by the particular vote, and joining the resulting array back together as a string.

The following is the CGI program to implement the processes previously described:

```
#!/usr/local/bin/perl
# conelec1.cgi
# This CGI program processes the consumer electronics survey form
# and updates the file that stores the survey data, survdat.dat
use CGI ":standard";
use Fcntl qw(:DEFAULT :flock);

# error - a function to produce an error message for the client
#         and exit in case of input/output errors
sub error {
    print start_html();
    print "Error - input/output error in conelec1.pl <br/>";
    print end_html();
    exit(1);
}

#>>> Begin the main program
#>>> Get the form values
my($age, $gender, $vote) = (param("age"), param("gender"),

#>>> Produce the header for the reply page - do it here so error
#>>>  messages appear on the page
print header;

#>>> Set $index to the line index of the current vote
$index = $age + $gender;

#>>> Open and lock the survey data file
open(SURVDAT, "+<survdat.dat") or error();
flock(SURVDAT, LOCK_EX) or error();

#>>> Read the survey data file, unlock it, and close it
for ($count = 0; $count <= 7; $count++) {
   chomp($file_lines[$count] = <SURVDAT>);
}

#>>> Split the line into its parts, increment the chosen device,
#>>>  and put it back together again
@file_votes = split / /, $file_lines[$index];
```

```
$file_votes[$vote]++;
$file_lines[$index] = join(" ", @file_votes);

#>>> Rewind the file for writing
seek(SURVDAT, 0, 0) or error();

#>>> Write out the file data and close it (which also unlocks it)
foreach $line (@file_lines) {
    print SURVDAT "$line\n";
}

close(SURVDAT);

#>>> Build the web page to thank the survey participant
print start_html("Thankyou");
print "Thank you for participating in our survey <br/> <br/> \n";
print end_html;
```

We now can develop the program, named `conelec2.cgi`, which produces the current status of the voting in the consumer electronics preference survey. Because there are vote totals in 56 different categories, it is best that the results are displayed in a table. Building this table in XHTML would be tedious without `CGI.pm`, and even with this module it is nontrivial.

Tables with `CGI.pm` are introduced through the following example: Suppose you want a CGI program to return a table of sales figures for the sales personnel Mary, Freddie, and Spot, including numbers for each of the five workdays in a week. The people will be the rows of the table; the days of the week will be the columns. Assume the sales figures are in three arrays, `@marysales`, `@freddiesales`, and `@spotsales`, each of which has five elements. The column titles are `Salesperson` (for the title row), `Monday`, `Tuesday`, `Wednesday`, `Thursday`, and `Friday`. The row titles for the three arrays are `Mary`, `Freddie`, and `Spot`.

The `table` function creates an entire table, which is specified with its parameters. If the table is to have a border, that is specified with the attribute `border`. The table's caption, if there is one, is created with the `caption` function:

```
caption("Sales Figures");
```

Each row of a table is created by a call to `Tr`. This shortcut is spelled with an uppercase first letter because, if it were not, it would be mistaken for the Perl transliterate operator, `tr`. The other parameters for `Tr` specify the row data of the table. The title row is built with a call to `th`, and the data rows are built with calls to `td`. All three functions, `Tr`, `th`, and `td`, take either references or arrays as parameters. Therefore, if a literal list is to be passed, it must be constructed

with brackets. If a parameter is an array, its name must be backslashed in the actual parameter list to specify its address.

Consider the following examples of the `th` shortcut function:

```
print th(['apples', 'oranges', 'grapes']);
```

produces

```
<th> apples </th>
<th> oranges </th>
<th> grapes </th>
```

and

```
@days = ('Saturday', 'Sunday');
print th(\@days);
```

produces

```
<th> Saturday </th>
<th> Sunday </th>
```

The `td` function is exactly like the `th` function.

In many cases, we want to build a table row that consists of a `th` element and several `td` elements and treat the whole thing as a single parameter to `Tr`. To treat these as a single row string, the returned values of the calls to `td` and `th` can be catenated together. For example, the code

```
print th('9:15').td('French', '', 'French', '');
```

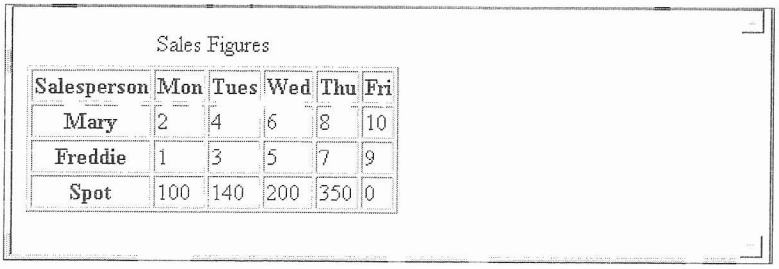
produces

```
<th> 9:15 </th> <td> French </td> <td> </td>
                <td> French </td> <td> </td>
```

Let us now consider a complete table specification.

```
# easy_table.pl
table({-border => "border"},
      caption("Sales Figures"),
       Tr(
         [th(["Salesperson", "Mon", "Tues", "Wed",
              "Thu", "Fri"]),
          th("Mary").td(\@marysales),
          th("Freddie").td(\@freddiesales),
          th("Spot").td(\@spotsales),
       ]
      )
     );
```

The browser display of the XHTML produced by this call to `table` is shown in Figure 9.6.



Sales Figures

Salesperson	Mon	Tues	Wed	Thu	Fri
Mary	2	4	6	8	10
Freddie	1	3	5	7	9
Spot	100	140	200	350	0

Figure 9.6 A table produced with `table`

Rows can have horizontal and vertical alignment attributes, specified by `align` and `valign`, respectively. These can be specified as the first parameter in the call to `Tr`.

Now we can get back to the development of the second survey-handling CGI program, `conelec2.cgi`. The basic tasks of `conelec2.cgi` are as follows:

1. Open the file and read the lines of the file into an array of strings.
2. Split the first four rows (responses from females) into arrays of votes for the four age groups.
3. Unshift row titles into the vote rows (making them the first elements).
4. Create the column titles row with `th` and put its address in an array.
5. Use `td` on each row of votes.
6. Push the addresses of the rows of votes onto the row address array.
7. Create the table using `Tr` on the array of row addresses.
8. Repeat steps 2 to 7 for the last four rows of data (responses from males).

Because the rows of the two result tables are nearly the same for the two tables, they are built with a subprogram, `make_rows`. This function takes one parameter, which is `0` for the female results table and `4` for the male results table. `make_rows` builds the table rows in the `@rows` array, which is globally accessed by the function and the calling program. The following is the CGI program to produce the vote totals tables:

```
#!/usr/local/bin/perl -w

# conelec2.cgi - display the survey results

#>>> make_rows - a subprogram to make the rows of an output table
sub make_rows {
    my $index = $_[0];

#>>> Split the input lines for females into age arrays
    @age1 = split(/ /, $vote_data[$index]);
```

```
    @age2 = split(/ /, $vote_data[$index + 1]);
    @age3 = split(/ /, $vote_data[$index + 2]);
    @age4 = split(/ /, $vote_data[$index + 3]);

#>>> Add the row titles to the age arrays
    unshift(@age1, "10-25");
    unshift(@age2, "26-40");
    unshift(@age3, "41-60");
    unshift(@age4, "Over 60");

#>>> Create the column titles in HTML by giving their address to the
#>>> th function and storing the return value in the @rows array
    @rows = th(\@col_titles);

#>>> Now create the data rows with the td function
#>>>  and add them to the row addresses array
   push(@rows, td(\@age1), td(\@age2), td(\@age3), td(\@age4));
}  #>>> end of the make_rows subprogram

#>>> error - a function to produce an error message for the client
#>>>         and exit in case of open errors
sub error {
    print start_html;
    print "Error - input/output error in conelec2.pl <br/>";
    print end_html;
    exit(1);
}  #>>> end of the error subprogram

use CGI qw(:standard);

#>>> Make the column titles array
@col_titles = ("Age Group", "Conventional TV", "HDTV", "VCR",
               "CD player", "MiniCD player/recorder", "DVD player",
               "Other");
print header;

#>>> Open and read the survey data file
open(SURVDAT, "<survdat.dat") or error();
@vote_data = <SURVDAT>;

#>>> Create the beginning of the result Web page
print start_html("Survey Results");
print h2("Results of the Consumer Electronics Purchasing Survey");
print "<br />";

#>>> Create the rows of the female survey results table
```

```
make_rows(0);

#>>> Create the table for the female survey results
#>>>  The address of the array of row addresses is passed to Tr
print table({-border => "border"},
            caption(h3("Survey Data for Females")),
            Tr(\@rows)
           );

#>>> Create the rows for the male results table
make_rows(4);

#>>> Create the table for the male survey results
#>>>  The address of the array of row addresses is passed to Tr
print "<br /><br />";
print table({-border => "border"},
            caption(h3("Survey Data for Males")),
            Tr(\@rows)
           );

print end_html;
```

Figure 9.7 shows the display of the table of voting results constructed by `conelec2.cgi` after some voting has occurred.

Results of the Consumer Electronics Purchasing Survey

Survey Data for Females

Age Group	Conventional TV	HDTV	VCR	CD player	MiniCD player/recorder	DVD player	Other
10-25	3	0	0	2	0	0	1
26-40	0	2	0	0	0	0	0
41-60	0	0	2	0	0	0	0
Over 60	0	0	0	0	0	0	0

Survey Data for Males

Age Group	Conventional TV	HDTV	VCR	CD player	MiniCD player/recorder	DVD player	Other
10-25	1	3	0	0	0	0	0
26-40	0	0	0	0	1	0	0
41-60	0	0	0	0	2	3	0
Over 60	0	0	0	2	0	0	0

Figure 9.7 Survey results page

Debugging a CGI program can be difficult. In many cases, errors are reported to the user as server errors, with the error number 500, which provides no clue as to what happened. The best course of action in such cases is to find out from the system administrator the location of the server error log. On UNIX systems the file is often named `error_log`; on Windows and OS/2 systems, it is often `error.log`. Once you determine the location and name of the server error log file, examine the most recent errors, and find the error message that is yours (on UNIX systems, this is done with the `tail` command). The error log information should help determine the problem.

9.6 Cookies

A *session* is the time span during which a browser interacts with a particular server. A session begins when a browser becomes connected to a particular server. It ends when the browser ceases to be connected to that server because either it becomes connected to a different server or it is terminated. The HTTP protocol is essentially stateless—it includes no means to store information about a session that is available to a subsequent session. However, there are a number of different reasons why it is useful for the server to be capable of connecting a request made during a session to the other requests made by the same client during that session, as well as previous and subsequent sessions. Many Web sites now create profiles of clients by remembering which parts of the site are perused. Later sessions can use such profiles to target advertising to the client according to the client's past interests. Also, if the server recognizes a request as being from a client who has made an earlier request from the same site, it is possible to present a customized interface to that client. These situations require that information about clients is accumulated and stored.

Cookies provide a general approach to storing information about sessions on the browser system itself. The server is given this information when the browser makes subsequent requests for Web resources from the server. Cookies allow the server to present a customized interface to the client. They also allow the server to connect requests from a particular client to previous requests, thereby connecting sequences of requests into a session.

A *cookie* is a small object of information consisting of a name and a textual value. A cookie is created by some software system on the server, such as a CGI program. Every HTTP communication between a browser and a server includes a header, which stores information about the message. A message from a browser to a server is a request; a message from a server to a browser is a response. The header part of an HTTP communication can include cookies. So, every request sent from a browser to a server, and every response from a server to a browser, can include one or more cookies.

At the time it is created, a cookie is assigned a lifetime. When the time a cookie has existed reaches its associated lifetime, the cookie is deleted from the browser's host machine. Every browser request includes all of the cookies its host machine has stored that are associated with the Web server to which the

request is directed. Only the server that created a cookie can ever receive the cookie from the browser, so a particular cookie is information that is exchanged exclusively between one specific browser and one specific server. Because cookies are stored as text, the browser user can view, alter, or delete them at any time.

Because cookies allow servers to record browser activities, they are considered by some to be a privacy concern. Accordingly, browsers allow the client to change the browser setting to refuse to accept cookies from servers. This is clearly a drawback of using cookies—the clients that reject them render them useless.

The `CGI.pm` module includes support for cookies in Perl, primarily through the `cookie` function. The `cookie` function serves both to create cookies and to retrieve existing cookies from the HTTP header of a request. The form of a call to `cookie` to create a cookie is as follows:

```
cookie(-name => a_cookie_name,
       -value => a_value,
       -expires => a_time_value)
```

The `cookie` function can take three more parameters, but they are not discussed here. The cookie name can be any string. The value can be any scalar value, including references to arrays and hashes. These last two allow the creation of multiple cookies with one call. The `expires` value, which specifies the lifetime of the cookie, can be expressed in many different units. For example, `+3d` specifies three days. The other units are `s` for seconds, `m` for minutes, `h` for hours, `M` for months, `y` for years, and `now` for right now. A negative value for `expires` effectively kills the cookie.

A cookie must be placed in the HTTP header at the time the header is created. With `CGI.pm`, this means when the `header` function is called. This is done by passing the cookie as a parameter to `header`, as in the following example:

```
header(-cookie => $my_cookie);
```

When the `cookie` function is called with no parameters, it returns a hash of all of the cookies in the HTTP header of the current request. To retrieve the value of one cookie, the `cookie` function is called with the name of the cookie. For example, the following call:

```
$age = cookie('age');
```

gets the value of the cookie named `age`.

To display all of the cookies, both names and values, in a CGI program, we could use the following:

```
print "Cookie Name \t Cookie Value <br  />";
foreach $name (keys cookie()) {
    print "$name \t cookie($name) <br />";
}
```

Suppose we want to provide a greeting to all visitors, including a message giving the day of the week, month, and day of the month of their last visit. To do

this, we could use a cookie to record the time and date of each visit. For the first visit by a client, a message could provide a first-time greeting.

For our program, we need to get the current date. The Perl `time` function returns the current time in seconds since January 1, 1970. The `localtime` function is more useful for our problem. It calls `time` and converts the number of seconds into nine values. For example, consider the following call to `localtime`:

```
($sec, $min, $hour, $mday, $mon, $year, $wday, $yday,
 $isdst) = localtime;
```

This statement sets the nine values for the current time and date, corrected for the time zone in which the Web server computer is installed. The first three scalars in the list have obvious meanings. The others have the following meanings: `$mday` is the day of the month; `$mon` is the month, coded as `0` to `11`; `$year` is the number of years since 1900; `$wday` is the day of the week, coded as `0` to `6`, where `0` is Sunday; `$yday` is the day of the year; and `$isdst` is a Boolean that specifies whether the given time is in daylight savings time.

The following example displays all of the nine values returned by `localtime`:

```
# time_date.pl
#  Input: None
# Output: The nine values returned by localtime
#

($sec, $min, $hour, $mday, $mon, $year, $wday, $yday,
  $isdst) = localtime;

print "\$sec = $sec\n";
print "\$min = $min\n";
print "\$hour = $hour\n";
print "\$mday = $mday\n";
print "\$mon = $mon\n";
print "\$year = $year\n";
print "\$wday = $wday\n";
print "\$yday = $yday\n";
print "\$isdst = $isdst\n";
```

The output of `time_date.pl` is as follows:

```
$sec = 43
$min = 20
$hour = 10
$mday = 19
$mon = 2
```

```
$year = 105
$wday = 6
$yday = 77
$isdst = 0
```

For days of the week and month, we often want the names rather than numbers. This conversion is easy in Perl. The names can be put in a list, which can be subscripted by the value returned from `localtime`. For example, the following statement sets `$day_of_week` to the name of the current day of the week:

```
$day_of_week = (qw(Sunday Monday Tuesday Wednesday
                   Thursday Friday Saturday))[(localtime[6]];
```

The subscript 6 is that of the day of the week (`$wday`).

The CGI program for our problem of creating a greeting for visitors with the time and date will do the following:

1. Get the cookie named `last_time`.
2. Get the current day of the week, month, and day of the month and put them in a cookie named `last_time`.
3. Put the cookie in the header of the return document.
4. If there was no existing cookie, produce a welcome message for the first-time visitor.
5. If there was a cookie, produce a welcome message that includes the previous day of the week, month, and day of the month.

The following is the Perl program to implement these actions:

```
#!/usr/bin/perl
# day_cookie.pl
#  A CGI-Perl program to use a cookie to remember the
#   day of the last login from a user and display it when run

use CGI ":standard";

#>>> Get the existing day cookie, if there was one
@last_day = cookie('last_time');

#>>> Get the current date and make the new cookie
$day_of_week = (qw(Sunday Monday Tuesday Wednesday Thursday
                   Friday Saturday)) [(localtime)[6]];
$month = (qw(January February March April May June July
             August September October November December))
          [(localtime)[4]];
$day_of_month = (localtime)[3];
@day_stuff = ($day_of_week, $day_of_month, $month);
```

```
$day_cookie = cookie(-name => 'last_time',
                     -value => \@day_stuff,
                     -expires => '+5d');

#>>> Produce the return document
#>>> First, put the cookie in the new header
print header(-cookie => $day_cookie);
print start_html('This is day_cookie.pl');

#>>> If there was no day cookie, this is the first visit
if (scalar(@last_day) == 0) {
    print "Welcome to you on your first visit to our site <br />";
}

#>>> Otherwise, welcome the user back and give the date of the
#>>>  last visit
else {
    ($day_of_week, $day_of_month, $month) = @last_day;
    print "Welcome back! <br /> ",
          "Your last visit was on ",
          "$day_of_week, $month $day_of_month <br />";
}
print end_html;
```

Figure 9.8 shows a browser display of a document returned by `day_cookie.pl`.

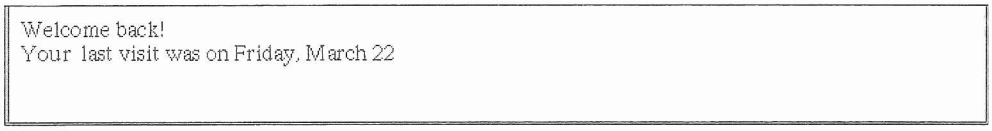


Figure 9.8 A document returned by `day_cookie.pl`

Summary

CGI is the interface between an XHTML document being displayed by a browser and a program that resides on the server. An XHTML document can specify a call to a CGI program with an anchor tag that includes the Internet address of the program. CGI programs can also be called as a side effect of a *Submit* button being clicked. A CGI program communicates with the XHTML document that called it by sending an XHTML document back to the browser

through the server. The XHTML tags in this document, as well as their content, are created by the CGI program through standard output.

XHTML forms are sections of documents that contain widgets, which are used to collect input from the user. The data specified in a form can be sent to the CGI program in either of two methods, `get` or `post`. The `get` method is fine for forms with relatively few widgets. It is also good for sending parameters to a CGI program when a form is not involved. The `post` method is more general because it has no limitation on the number of widgets included in the form. The coded data from a form is called a query string. With `get`, the query string is attached to the URL of the CGI program.

The values specified in a form are together called form data. When the *Submit* button is clicked, the query string, which is an encoded version of the form data, is created from the form data and sent to the server. Each widget that has a value is included in the query string. The format of each of these is a name/value pair, separated by an equal sign. These assignments are separated by ampersands. All special characters are coded by using a percent sign followed by the ASCII code for the character, in hexadecimal.

The `CGI.pm` module provides convenient aids to writing CGI programs in Perl. The shortcut functions of `CGI.pm` produce the tags after which they are named. Attribute values are passed to functions in `CGI.pm` using the form of the elements in a hash literal. The attribute names are preceded by minus signs. Parameters to functions follow the attribute values, if there are any. Tags and their attributes distribute over a list parameter. This list parameter must actually be the address of a list or array. Tables are built with `table`, table rows are built with `Tr`, table headings are built with `th`, and table data is created with `td`. `Tr`, `th`, and `td` allow references as parameters. The `header` function produces the first two lines of the return XHTML document. The `start_html` function produces the `<head>`, `<title>`, `</title>`, `</head>`, and `<body>` tags. Its parameter is used for the content of `<title>`. The `param` function takes a name as its parameter. It returns the value from the query string of that name. The `end_html` function produces the closing tags for `<body>` and `<html>`.

Cookies are small pieces of textual information that are exchanged between Web servers and browsers. They originate from server-based programs but are stored on browser systems. They are used to store information about a client between sessions the browser has with specific servers.

Review Questions

9.1 What are three categories of operations that are essential in Web documents but that cannot be done with XHTML?

9.2 On what system, the client or the server, do CGI programs reside when they are executed?

9.3 What forms are legal for the response from a CGI program?

9.4 What is the most common way for a client to provide information to the server?

9.5 What are the actions of the *Submit* button in a form?

9.6 How is the CGI program that processes the data provided by a form specified in the form?

9.7 Must a CGI program that processes a form reside on the server that provided the form to the client?

9.8 What part of the HTTP header is always provided as part of the response of a CGI program?

9.9 What is form data?

9.10 What is a query string?

9.11 How is a query string transmitted to the server with the `get` method?

9.12 How is a query string transmitted to the server with the `post` method?

9.13 What is the format of a query string that has multiple widget data values?

9.14 Why are special characters coded in query strings?

9.15 What is the purpose of the shortcuts in `CGI.pm`?

9.16 If both content and attribute values are passed to a shortcut, what is the format of these parameters?

9.17 Explain how tags and their attributes are distributed over the parameters to a shortcut function.

9.18 How are arrays passed to shortcut functions?

9.19 Why should a file to be read or written by a CGI program be locked against multiple simultaneous operations?

9.20 Where are cookies stored?

9.21 What is the form of the value of a cookie?

Exercises

9.1 Write an XHTML document that contains an anchor tag that calls a CGI program. Write the called CGI program, which returns a randomly chosen greeting from a list of five different greetings. The greetings must be stored as constant strings in the program. A random number between 0 and 4 can be computed with these lines:

```
srand;  # Sets the seed for rand
$number = int(rand 4);  # Computes a random integer 0-4
```

9.2 Modify the CGI program for Exercise 9.1 to count the number of visitors and display that number for each visitor.

9.3 Write an XHTML document to create a form with the following capabilities:

a. A text widget to collect the user's name

b. Four checkboxes, one each for the following items:

i. Four 100-watt light bulbs for $2.39

ii. Eight 100-watt light bulbs for $4.29

iii. Four 100-watt long-life light bulbs for $3.95

iv. Eight 100-watt long-life light bulbs for $7.49

c. A collection of three radio buttons that are labeled as follows:

i. Visa

ii. MasterCard

iii. Discover

9.4 Write a Perl CGI program that computes the total cost of the ordered light bulbs from Exercise 9.3, after adding 6.2 percent sales tax. The program must inform the buyer of exactly what was ordered, in a table.

9.5 Revise the survey sample CGI program of this chapter to make the table that displays the results of the survey have consumer electronics devices as its rows rather than its columns.

9.6 Revise the survey sample CGI program of this chapter to record the number of votes so far in the data file and display that count every time a vote is submitted or a survey result is requested. Also, change the output table so that its data is a percentage of the total votes for the particular age category.

9.7 Write an XHTML document to create a form that collects favorite popular songs, including the name of the song, the composer, and the performing artist or group. This document must call one CGI program when the form is submitted and another to request a current list of survey results.

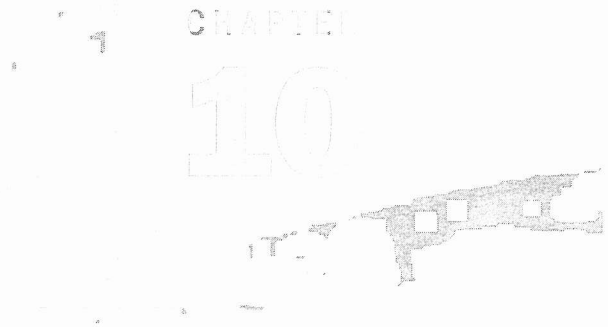
9.8 Write a CGI program that collects the data from the form of Exercise 9.7 and writes it to a file.

9.9 Write a CGI program that produces the current results of the survey of Exercise 9.7.

9.10 Write an XHTML document to provide a form that collects names and telephone numbers. The phone numbers must be in the format ddd-ddd-dddd. Write a CGI program that checks the submitted telephone number to be sure it conforms to the required format and then returns a response that indicates whether the number was correct.

9.11 Modify the `day_cookie.pl` program to have it return the number of months, days, hours, and minutes since the last visit by the current client.

9.12 Write a CGI program that collects the name of every visitor (in a form text element). The program must create a cookie to save the visitor's name and include a brief personalized greeting to every repeat visitor.



Servlets and Java Server Pages

10.1 Overview of Servlets
10.2 Servlet Details
10.3 A Survey Example
10.4 Storing Information on Clients
10.5 Java Server Pages
Summary • Review Questions • Exercises

This chapter discusses Java server-based software, specifically servlets and Java Server Pages. First, servlets are introduced, including their general structure and common uses. The servlet methods for handling `GET` and `POST` HTTP requests are then discussed. This includes the objects used for carrying information between the client and the servlet on the server. Four complete examples are provided to illustrate servlets.

Next, the chapter describes two servlet approaches to storing information about clients: cookies, which were discussed in Chapter 9, "Using Perl for CGI Programming," and session tracking. Each of these is illustrated with a complete servlet example.

Lastly, Java Server Pages are introduced. The focus is on the JSP Standard Tag Library and the Expression Language. Java beans and scriptlets are not discussed, although they are part of Java Server Pages. Accessing values for the various kinds of form components is discussed, using implicit variables and action elements.

10.1 Overview of Servlets

Simply put, a *servlet* is a server-side version of an applet. Applets are executed on a client system under the control of the browser, after being requested by the XHTML document being displayed by the browser. A servlet is a compiled Java class, an object of which is executed on the server system when requested by the XHTML document being displayed by the browser. The servlet class is instantiated when the Web server begins execution. The execution of a servlet is managed by a *servlet container*. The servlet container may run in the same process as the Web server, in a different process on the server host machine, or even on a different machine. The servlet request and response processes are supported with the HTTP protocol, so the servlet container must implement the HTTP specification. A servlet container might also define and enforce security restrictions on the execution of its servlets. Servlet containers are sometimes called *servlet engines*.

When an HTTP request is received by a Web server, the Web server examines the request. If a servlet must be called, the Web server passes the request to the servlet container. The container determines which servlet must be executed, makes sure it is loaded, and calls it. A servlet call passes two parameter objects: one with the request and one for the response. The servlet receives the input data associated with the request through the request object. This may include form data as well as the identity of the requesting client. As the servlet handles the request, it dynamically generates an XHTML document as its response, which is returned to the server through the response object parameter. The process of handling the request (by the servlet) is accomplished in part by calling methods on the request and response objects. When finished, the servlet container returns control to the Web server.

Servlets are often used as alternatives to CGI programs to dynamically generate responses to browser requests. They are also used as alternatives to server extensions such as Apache modules, which users can write and add to an Apache server to extend its capabilities. Servlets have some potential advantages over CGI for providing server-based computation and server extensions. Because servlets continue to run, they are capable of saving status information, whereas CGI programs start, execute, and stop, providing no way to save such information except on the server's disk. Also, there are the general advantages that Java has over the languages typically used for CGI, such as Perl, in the areas of software reliability and maintainability.

When servlets were introduced, they were faster than CGI programs because, at that time, every execution of a CGI program required a new process to be spawned. Newer versions of Perl, such as `mod_perl`, avoid the new process creation for CGI programs, thereby eliminating much of the speed advantage of servlets.

10.2 Servlet Details

All servlets either implement the `Servlet` interface or extend a class that implements `Servlet`. The `Servlet` interface, which is defined in the `javax.servlet` package, declares the methods that manage servlets and their interactions with clients. The author of a servlet must provide definitions of these methods.

The `GenericServlet` class is a predefined implementation of the `Servlet` interface. The `HttpServlet` class is a predefined extension to `GenericServlet`.[1] Most user-written servlets are extensions to `HttpServlet`.

In addition to the `Servlet` interface, the `javax.servlet` package contains several other interfaces required for implementing servlets. The `ServletRequest` and `ServletResponse` interfaces encapsulate the communication from the client to the servlet and from the servlet back to the client, respectively. The `ServletRequest` interface provides servlet access to `ServletInputStream`, through which input from the client flows. The `ServletResponse` interface provides servlet access to `ServletOutputStream` and also provides a method used to send information, usually in the form of an XHTML document, back to the client.

Every subclass of `HttpServlet` must override at least one of the methods of `HttpServlet`, the most common of which are shown in Table 10.1.

Table 10.1 Commonly used methods of `HttpServlet`

Method	Purpose
`doGet`	To handle HTTP `GET` requests
`doPost`	To handle HTTP `POST` requests
`doPut`	To handle HTTP `PUT` requests
`doDelete`	To handle HTTP `DELETE` requests

The `doGet`, `doPost`, `doPut`, and `doDelete` methods are called by the server. The HTTP `PUT` request allows a client to send a file to be stored on the server. The HTTP `DELETE` request allows a client to delete a document or Web page from the server. In many cases, users are not allowed to add files to the server or delete files that are stored on the server. We focus on `doGet` and `doPost` in this chapter because they are the most frequently used of the `HttpServlet` methods.

1. There are other extensions to `GenericServlet` (for example, to handle other protocols such as the Simple Object Access Protocol [SOAP]).

The protocol of the `doGet` method is as follows:

```
protected void doGet (HttpServletRequest request,
                      HttpServletResponse response)
          throws ServletException, java.io.IOException
```

`ServletException` is a subclass of `Exception` that serves as a wrapper for every kind of general servlet problem. `java.io.IOException` can be thrown for the usual reasons. In the protocol model above, `request` and `response` are the names we have chosen to be the reference variables for the request and response objects, respectively. The `HttpServletRequest` object parameter, `request`, contains the client request; the `HttpServletResponse` object parameter, `response`, provides the means to communicate the response that the servlet sends back to the client.

The protocol of the `doPost` method is the same as that of `doGet`.

Servlet output to the requesting client is created by defining a `PrintWriter` object through the `HttpServletResponse` object, using the `getWriter` method. The `PrintWriter` object provides a collection of methods, such as `println`, that sends response XHTML code to the client, through the response object. The `PrintWriter` object corresponds to the standard output stream from a Perl CGI program.

Before the `PrintWriter` object is created, it is essential that the content type of the return document be set. This is done with the `setContentType` method of the `HttpServletResponse` object, as is shown in the following example:

```
response.setContentType("text/html");
```

Following this method call, the `PrintWriter` object can be created with the following:

```
PrintWriter servletOut = response.getWriter();
```

Now the `println` method of the `servletOut` object can be used to generate the XHTML markup of the document to be returned to the requesting client.

We are now ready to look at a complete servlet example. This first example servlet simply responds to a call from a form that uses the `GET` method. The form sends no data and requires no processing. So, the only action of the servlet is to produce an XHTML document with a message to indicate that the call was received. The call to the servlet, which appears in the XHTML form tag, specifies the location of the servlet as the value of the form tag's `action` attribute. The servlet examples in this chapter were all run on the same machine as the browser, so the location is `localhost` (this machine), the servlet directory, and the servlet name. The following is the XHTML document that will call the servlet. Figure 10.1 shows the display created by `tstGreet.html`.

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- tstGreet.html
     Used to test the servlet, Greeting
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Test greeting </title>
  </head>
  <body>
    <form action = "http://localhost/servlet/Greeting"
          method = "get">
      <p>
        Press the button to enact the servlet
        <input type = "submit" value = "Enact Servlet" />
      </p>
    </form>
  </body>
</html>
```

Figure 10.1 Display of `tstGreet.html`

Notice that the value of the action attribute of the form tag uses the name of the servlet class, not the name of the file that stores the servlet's code.

The `Greeting` servlet class extends `HttpServlet` and implements the `doGet` method, which produces the XHTML response to the browser call. Following is a listing of the `Greeting` servlet:

```
/*  Greeting.java
    A servlet to illustrate a simple GET request
    */
import javax.servlet.*;
import javax.servlet.http.*;
```

```
import java.io.*;

public class Greeting extends HttpServlet {
    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException {
        PrintWriter returnHTML;
        response.setContentType("text/html");
        returnHTML = response.getWriter();
        returnHTML.println("<html><head><title>");
        returnHTML.println("A simple GET servlet");
        returnHTML.println("</title></head><body>");
        returnHTML.println(
            "<h2> This is your servlet answering </h2>");
        returnHTML.println("</body></html>");
        returnHTML.close();
    }
}
```

Figure 10.2 shows the response from the `Greeting` servlet.

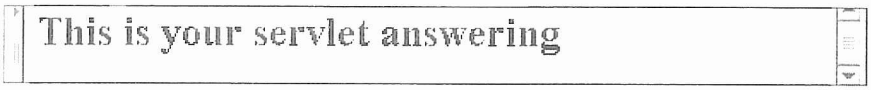


Figure 10.2 The response from the `Greeting` servlet

One popular way of running servlets is the Tomcat servlet container, which is available free from the Apache Group at `http://www.apache.org/jakarta`. Tomcat can run as a standalone servlet container or as part of another Web server.

10.3 A Survey Example

The next servlet example is more complicated and interesting than the `Greeting` servlet. Chapter 9 includes an example in which a form is used to gather responses for a survey of potential purchasers of consumer electronics products. It uses two Perl CGI programs to process the form data: one to collect responses and one to produce the current results of the survey. We now reformulate that example into a simpler form and use a single servlet to collect the responses and to produce the current results. The XHTML document for the new survey follows:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- conelec2.html
     A simplified version of the consumer electronics
     survey form from chapter 10.
     This version uses a servlet for processing the form data
   -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Consumer Electronics Purchasing Survey </title>
  </head>
  <body>
    <form action = "http://localhost/servlet/Survey"
          method = "post">
      <h2> Welcome to the Consumer Electronics Purchasing Survey
      </h2>
      <p />
      <h4> Your Gender: </h4>
      <p>
        <label>
          <input type = "radio" name = "gender" value = "female"
                 checked = "checked" />
          Female <br />
        </label>
        <label>
          <input type = "radio" name = "gender" value = "male" />
          Male <br /> <br /> <br />
        </label>
      </p>
      <h4> Your Next Consumer Electronics Purchase will be: </h4>
      <p>
        <label>
          <input type = "radio" name ="vote" value="0" />
          Conventional TV <br />
        </label>
        <label>
          <input type = "radio" name ="vote" value="1" />
          HDTV <br />
        </label>
        <label>
          <input type = "radio" name ="vote" value="2" />
          VCR <br />
```

```
        </label>
        <label>
          <input type = "radio" name ="vote" value="3" />
          CD player <br />
        </label>
        <label>
          <input type = "radio" name ="vote" value="4" />
          Mini CD player/recorder <br />
        </label>
        <label>
          <input type = "radio" name ="vote" value="5" />
          DVD player <br />
        </label>
        <label>
          <input type = "radio" name="vote" value="6"
                 checked = "checked" />
          Other <br /> <br />
        </label>
        <input type = "submit" value = "Submit Order" />
        <input type = "reset" value = "Clear Order Form" />
      </p>
    </form>
  </body>
</html>
```

Figure 10.3 shows the display of `conelec2.html`.

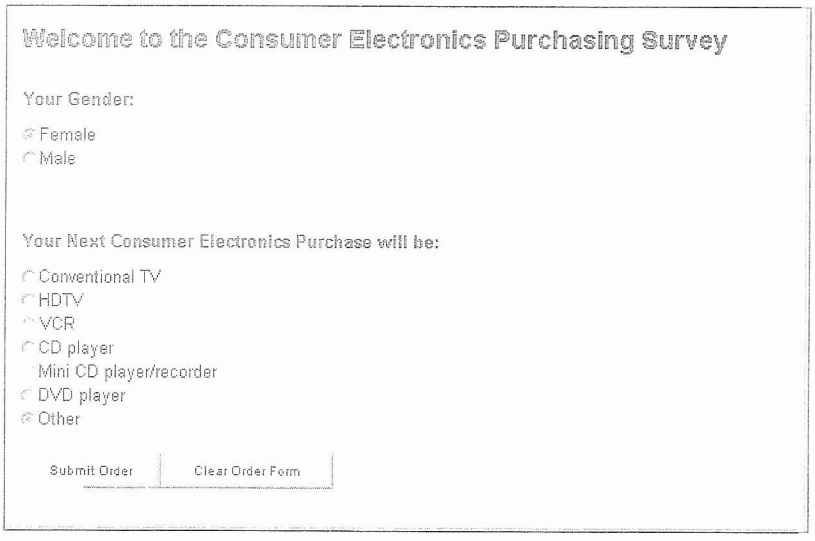


Figure 10.3 Display of `conelec2.html`

Because the servlet that processes the form in this page must accumulate the results of the survey, it must create and use a file. The first time the form is submitted, the file must be created and written. For all subsequent submissions, the file is opened, read, and rewritten. The servlet will produce the current vote totals for every client that submits a form. The survey results will be just the totals for each kind of product.

The data stored in the vote totals file is an integer array of results. The approach used was to read and write the file using the `ObjectInputStream` and `ObjectOutputStream` objects, respectively. This is a simple way to write any object to a file. When used for input, the input data object is cast to an integer array. For file output, the array object is written directly to the stream.

On all calls to the servlet except the first, the servlet must read the current vote array from the file, modify it, and write it back to the file. On the first call, there is no need to read the file first because the call creates the first vote to be written to the file. The `ObjectInputStream` object used to read the file is created by a call to the `ObjectInputStream` constructor, passing an object of class `FileInputStream`, which is itself created by passing the file's program name to the `FileInputStream` constructor. All of this is specified with the following statement:

```
ObjectInputStream indat = new ObjectInputStream(
      new FileInputStream(File_variable_name));
```

In this statement, `indat` is defined as the program variable that references the input stream.

As was the case with the Perl CGI program in Chapter 9 that wrote to a server file, there can be concurrent accesses to the file used here. A servlet container can support multiple simultaneous executions of a servlet. To prevent corruption caused by concurrent accesses to the file, a `synchronized` clause can be used to enclose the file accesses. Whatever code that is in such a clause executes completely before a different execution is allowed to enter the clause.

The servlet accesses the form data with the `getParameter` method of the request object that was passed to the `doPost` method. This method takes a string parameter, which is the name of the form element. The string value of the parameter is returned. For example, if the form has an element named `address`, the following statement will put the value of the address form element in the variable `newAddress`:

```
newAddress = request.getParameter("address");
```

If the element whose name is sent to `getParameter` does not have a form value, `getParameter` returns `null`. Note that `getParameter` also works for values passed through the `GET HTTP` method, so it can be used in `doGet` methods.

Form values that are not strings must be converted because they are all passed as strings. So, if a form value is an integer number, it is passed as a string and must be converted back to an integer value in the servlet. In Java, this is done with the `parseInt` method, which is defined in the wrapper class for integers, `Integer`. For example, to get the integer value of a parameter that is passed as the form value of an element named `price`, the following could be used:

```
price = Integer.parseInt(request.getParameter("price"));
```

We can now discuss the specifics of the servlet for processing the survey form data. The data file stores an array of 14 integers, seven votes for female voters and seven votes for male voters. The actions of the servlet are described in the following pseudocode algorithm:

```
If the votes data file exists
  read the votes array from the data file
else
  create the votes array
Get the gender form value
Get the form value for the new vote
  and convert it to an integer
Add the vote to the votes array
Write the votes array to the votes file
Produce the return XHTML document that shows the
  current results of the survey
```

The servlet, `Survey`, that implements this process is as follows:

```
// Survey.java
// This servlet processes the consumer electronics survey
//  form, updating the file that stores the survey data
//  and producing the current total votes in the survey.
//  The survey data file, survdat.dat, is stored on the Web server.

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
public class Survey extends HttpServlet {
    public void doPost(HttpServletRequest request,
                       HttpServletResponse response)
        throws ServletException, IOException {
        int votes[] = null;
        int index;
        int vote;
        File survdat = new File("survdat.dat");
        String gender;
        String products[] = {"Conventional TV", "HDTV", "VCR",
                             "CD Player",
                             "Mini CD player/recorder",
                             "DVD player", "Other"};

// Set the content type for the response output and get a writer
        response.setContentType( "text/html" );
        PrintWriter servletOut = response.getWriter();
```

```
// Produce the head of the output document
        servletOut.println("<html><head>");
        servletOut.println(
            "<title> Return message </title></head><body>");

// Synchronize a block for the votes file access
        synchronized(this) {

// If the file already exists, read in its data
          try {
            if (survdat.exists()) {
              ObjectInputStream indat = new ObjectInputStream(
                              new FileInputStream(survdat));
              votes = (int []) indat.readObject();
              indat.close();
            }

// If the file does not exist (this is the first vote), create the
//  votes array
            else
              votes = new int[14];
          }
          catch (Exception e) {e.printStackTrace();}

// Get the gender of the survey respondee
          gender = request.getParameter("gender");

// Add the consumer electronics vote of the response to the
//  votes array
          vote = Integer.parseInt(request.getParameter("vote"));
          if (gender.equals("male")) vote += 7;
          votes[vote]++;

// Write updated votes array to disk
          ObjectOutputStream outdat = new ObjectOutputStream(
              new FileOutputStream(survdat));
          outdat.writeObject(votes);
          outdat.flush();
          outdat.close();
        }  //** end of the synchronized block

// Create the initial response information
        servletOut.println(
            "<h3>Thank you for participating in the");
        servletOut.println(" Consumer Electronics Survey</h3>");
        servletOut.println("<h4>Current Survey Results:</h4>");
```

```
// Create the total votes return information for female respondents
        servletOut.println("<h5>For Female Respondents </h5>");
        for (index = 0; index < 7; index++) {
            servletOut.print(products[index]);
            servletOut.print(": ");
            servletOut.println(votes[index]);
            servletOut.println("<br />");
        }

 // Create the total votes return information for male respondents
        servletOut.println("<h5>For Male Respondents </h5>");
        for (index = 7; index < 14; index++) {
            servletOut.print(products[index - 7]);
            servletOut.print(": ");
            servletOut.println(votes[index]);
            servletOut.println("<br />");
        }
        servletOut.close();
    }
}
```

Figure 10.4 shows the results of running the survey servlet after some survey responses have been received.

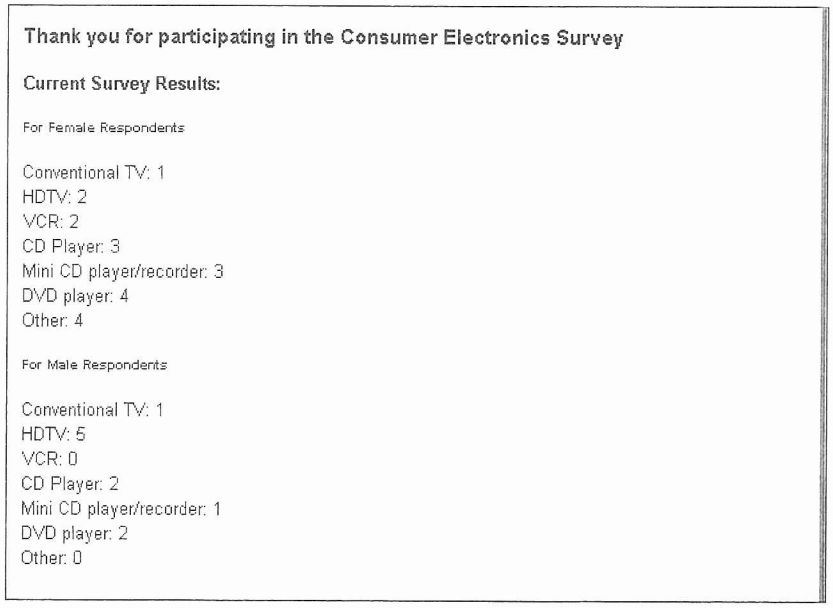


Figure 10.4 Results of the `Survey` servlet

10.4 Storing Information on Clients

Chapter 9 discussed cookies, which are used to store information about a Web client on the client's machine. The `javax.servlet` package provides the tools for creating and using cookies as well as a cookie alternative, session tracking.

10.4.1 Cookies

The following introduction to cookies also appears in Chapter 9, "Using Perl for CGI Programming." It is repeated here for those who have chosen to skip Chapter 9.

A *session* is the time span during which a browser interacts with a particular server. A session begins when a browser becomes connected to a particular server. It ends when the browser ceases to be connected to that server because either it becomes connected to a different server or it is terminated. The HTTP protocol is essentially stateless—it includes no means to store information about a session that is available to a subsequent session. However, there are a number of different reasons why it is useful for the server to be capable of connecting a request made during a session to the other requests made by the same client during that session, as well as previous and subsequent sessions. Many Web sites now create profiles of clients by remembering which parts of the site are perused. Later sessions can use such profiles to target advertising to the client according to the client's past interests. Also, if the server recognizes a request as being from a client who has made an earlier request from the same site, it is possible to present a customized interface to that client. These situations require that information about clients is accumulated and stored.

Cookies provide a general approach to storing information about sessions on the browser system itself. The server is given this information when the browser makes subsequent requests for Web resources from the server. Cookies allow the server to present a customized interface to the client. They also allow the server to connect requests from a particular client to previous requests, thereby connecting sequences of requests into a session.

A *cookie* is a small object of information, which consists of a name and a textual value. A cookie is created by some software system on the server. Every HTTP communication between a browser and a server includes a header, which stores information about the message. A message from a browser to a server is a request; a message from a server to a browser is a response. The header part of an HTTP communication can include cookies. So, every request sent from a browser to a server, and every response from a server to a browser, can include one or more cookies.

At the time it is created, a cookie is assigned a lifetime. When the time a cookie has existed reaches its associated lifetime, the cookie is deleted from the browser's host machine. Every browser request includes all of the cookies its host machine has stored that are associated with the Web server to which the request is directed. Only the server that created a cookie can ever receive the cookie from the browser, so a particular cookie is information that is exchanged

exclusively between one specific browser and one specific server. Because cookies are stored as text, the browser user can view, alter, or delete them at any time.

Because cookies allow servers to record browser activities, they are considered by some to be privacy concerns. Accordingly, browsers allow the client to change the browser setting to refuse to accept cookies from servers. This is clearly a drawback of using cookies—the clients that reject them render them useless.

10.4.2 Servlet Support for Cookies

On the server, a cookie associated with a servlet is an object of class `Cookie`; on a client, a cookie is just a text data value. It is good to keep these two uses of the term *cookie* distinct.

A Java cookie object has a collection of data members and methods. Among the most commonly used data members are those for storing the lifetime, or maximum age, of the cookie and for storing the cookie's name and value as strings, along with a comment, which is a string that can be used to explain the purpose of the cookie. The most commonly used `Cookie` methods are `setComment(String)`, `setMaxAge(int)`, `setValue(String)`, `getComment()`, `getMaxAge()`, `getName()`, and `getValue()`, all of whose purposes are obvious from their names.

A cookie object is created with the constructor for the `javax.servlet.http.Cookie` class. This constructor takes two parameters: the cookie name and the cookie value. For example, consider the following statement:

```
Cookie newCookie = new Cookie(gender, vote);
```

By default, a cookie exists from the time it is created until the current session ends, which is when the browser that started the session is terminated. If you want the cookie to exist past the end of the current session, you must use the `setMaxAge` method of `Cookie` to give the cookie a specific lifetime. The parameter to `setMaxAge` is the number of seconds, expressed as an integer literal. Because Java integers can have values up to a maximum of about two billion, cookies can have ages that range from one second to nearly 25,000 years. For example, the following method call gives `newCookie` a lifetime of one hour:

```
newCookie.setMaxAge(3600);
```

A cookie is attached to a response from a server with the `addCookie` method of the `HttpServletResponse` class. For example, the cookie `newCookie` can be added to the response object `myResponse` with the following statement:

```
myResponse.addCookie(newCookie);
```

Note that the cookie must be added to the response before any other part of the response is created, even the content type. When cookies are used, the sequence of response creation must be as follows:

1. Add any cookies to the response with `addCookie`.
2. Set the content type of the message with `setContentType`.
3. Get a response output stream with `getWriter`.
4. Place response information in the response stream with `print` or `println`.

Once again, remember that the cookie that a browser gets and stores is not a Java object—it has no methods; it is just some data.

The browser has little to do with cookies, at least directly. Browsers accept cookies, store them on the browser system, and return them to the server that created them with each `GET` or `POST` request to that server that occurs before the session ends or the cookie's lifetime ends. All of this is done implicitly by the browser.

A cookie that is sent from the browser to the server must be explicitly received by the server. In the case of a servlet, this is done with the `getCookies` method of `HttpServletRequest`. This method returns an array of references to `Cookie` objects. The following is an example of a cookie array declaration and a subsequent call to `getCookies`:

```
Cookie theCookies [];
...
theCookies = request.getCookies();
```

Whatever cookie processing is required can be done before the cookies are attached to the response and sent back to the browser.

10.4.3 An Example

We now consider an example of a ballot form that collects client votes in an election for the esteemed position of dogcatcher. The votes submitted through this form are recorded on the server by a servlet, which handles the form. This example uses a cookie to record, on the client, whether the voter has voted before, the objective being to prevent multiple votes from the same client. The survey form is presented with the XHTML document named `ballot.html`, which follows:

```
<?xml version = "1.0" encoding = "utf-8"?>
<!DOCTYPE html PUBLIC "-//w3c//DTD XHTML 1.1//EN"
 "http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">

<!-- ballot.html
     Presents a ballot to the user and calls
     the VoteCounter servlet for form handling
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head> <title> Ballot </title>
```

```
  </head>
  <body>
    <form action = "http://localhost/servlet/VoteCounter"
          method = "post">
      <h3> Please choose one candidate for dogcatcher </h3>
      <p>
        <input type = "radio" name = "vote" value = "Dogman" />
        Daren Dogman <br />
        <input type = "radio" name = "vote"
               value = "Taildragger" />
        Timmy Taildragger <br/>
        <input type = "radio" name = "vote" value = "Dogpile" />
        Don Dogpile <br />
      </p><p>
        <input type = "submit" value = "Submit ballot" />
      </p>
    </form>
  </body>
</html>
```

Figure 10.5 shows the display of the form described in `ballot.html`.

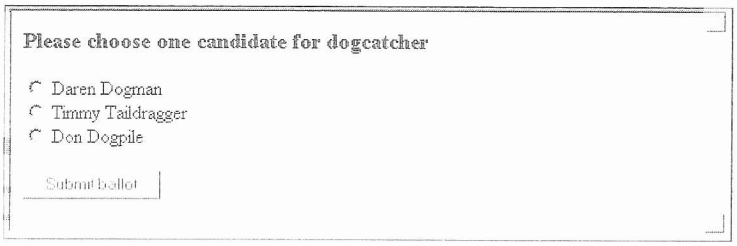


Figure 10.5 Display of `ballot.html`

The users of the ballot form can vote for one of three persons for dogcatcher. The form presents the three choices as radio buttons and includes a *Submit ballot* button. The `action` attribute of the form specifies that it be handled by the servlet `VoteCounter`, using the `POST` method.

The vote-counting servlet has several processing responsibilities. For each ballot (request) the servlet receives, it must first determine whether a vote was actually cast. If no vote was cast, it must send an XHTML document back to the client, asking the user to mark a vote and return the ballot form. It must also ensure that a voter does not vote twice, at least for some specified period of time. To do this, a cookie is returned to each voter. Each vote submission is checked to determine whether a cookie showing that the user has already voted came along with the ballot. If the ballot contains a vote—that is, if the form has

one of its radio buttons pressed—and the voter has not voted previously, the vote must be processed. Processing a vote means reading the vote totals file, updating it, and writing it back to disk storage. Finally, the servlet must produce the current vote totals for each legitimate voter, in the form of an XHTML document. The actions of the `VoteCounter` servlet are outlined in the following pseudocode algorithm:

```
If the form does not have a vote
  return a message to the client—"no vote"
else
  If the client did not vote before
    If the votes data file exists
      read in the current votes array
    else
      create the votes array
    end if
    update the votes array with the new vote
    write the votes array to disk
    make an "iVoted" cookie and add it to the response
    return a message to the client, including the new vote totals
  else
    return a message to the client—"Illegal vote"
  end if
end if
```

Two utility methods are used: a predicate method to determine whether the client has voted and a method to create the XHTML header text. The servlet code follows:

```
// VoteCounter.java
// This servlet processes the ballot form, returning a
// page asking for a new vote if no vote was made on the
// ballot. For legitimate ballots, the vote is added to
// the current totals, and those totals are presented to
// the user in a return page.
//  A cookie is returned to the voter, recording the fact
//  that a vote was received. The servlet examines all votes
//  for cookies to ensure that there is no multiple voting.
//  The voting data file, votesdat.dat, is stored on the Web
//  server.

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
```

```
public class VoteCounter extends HttpServlet {
    Cookie cookies[] = null;
    int index;
    PrintWriter servletOut;

    public void doPost(HttpServletRequest request,
                       HttpServletResponse response)
        throws ServletException, IOException {
        Cookie newCookie;
        int votes[] = null;
        String vote;
        File votesdat = new File("votesdat.dat");
        String candidates[] = {"Daren Dogman", "Timmy Taildragger",
                               "Don Dogpile"};

// Get cookies from the request
        cookies = request.getCookies();

// Check to see if there was a vote on the form
        vote = request.getParameter("vote");
        if (vote == null) {  // There was no vote

// Create the return page
            makeHeader(response);
            servletOut.println(
            "You submitted a ballot with no vote marked <br />");
            servletOut.println(
                "Please mark the ballot and resubmit");
        }       // end of if (vote == null)...
        else {  // There was a vote

// Check to see if this client voted before
            if (!votedBefore()) {

// No previous vote, so get the contents of the file (if the file
//  already exists)

// Synchronize block for file input/output
                synchronized(this) {
                    if (votesdat.exists()) {
                        ObjectInputStream indat =
                            new ObjectInputStream(
                            new FileInputStream(votesdat));

// We need the try/catch here because readObject can throw
//  ClassNotFoundException
```

```
                        try {
                            votes = (int []) indat.readObject();
                        }
                        catch(ClassNotFoundException problem) {
                           problem.printStackTrace();
                        }
                    }  //** end of if(votesdat.exists() ...

// If the file does not exist (this is the first vote), create the
//  votes array
                    else
                        votes = new int[3];

// Add the new vote to the votes array
                    if (vote.equals("Dogman"))
                        votes[0]++;
                    else if (vote.equals("Taildragger"))
                        votes[1]++;
                    else votes[2]++;

// Write updated votes array to disk
                    ObjectOutputStream outdat =
                       new ObjectOutputStream(
                       new FileOutputStream(votesdat));
                    outdat.writeObject(votes);
                    outdat.flush();
                    outdat.close();
                } //** end of synchronize block

// Attach a cookie to the response
                newCookie = new Cookie("iVoted", "true");
                newCookie.setMaxAge(5);  // Set to 5 for testing
                response.addCookie(newCookie);

// Write a response message
                makeHeader(response);
                servletOut.println("Your vote has been received");
                servletOut.println(
                    "<br /> <br /> Current Voting Totals:<br />");

// Create the total votes return information
                for (index = 0; index < 3; index++) {
                    servletOut.println("<br />");
                    servletOut.print(candidates[index]);
                    servletOut.print(": ");
                    servletOut.println(votes[index]);
```

```
                }
            }  // end of if (!votedBefore() ...
            else {  // The client voted before

// Write a response message
                makeHeader(response);
                servletOut.println(
                  "Your vote is illegal - you have already voted!");
                  }  // end of else clause - client voted before
        }  // end of else (there was a vote)

// Finish response document and close the stream
         servletOut.println("</body> </html>");
         servletOut.close();

    }  // end of doPost

//-----------------------------------------------------------------
// Method votedBefore - return true if the client voted before;
//  false otherwise

    boolean votedBefore() {
        if (cookies == null || cookies.length == 0)
            return false;
        else {

// Check the cookies to see if this user voted before
            for (index = 0; index < cookies.length; index++) {
                 if (cookies[index].getName().equals("iVoted")
                     && cookies[index].getValue().equals("true"))
                     return true;
            }  // end of for (index = 0; ...
            return false;
        }  // end of if (cookies == null ...
    }  // end of votedBefore
//-----------------------------------------------------------------
// Method makeHeader - get the writer and produce
// the response header

    void makeHeader(HttpServletResponse response)
             throws IOException {

// Set content type for response and get a writer
        response.setContentType("text/html");
        servletOut = response.getWriter();
```

```
// Write the response document head and the message
        servletOut.println("<html><head>");
        servletOut.println(
                "<title> Return message - </title></head><body>");
    }  // end of makeHeader
}  // end of VoteCounter
```

The outputs of the `VoteCounter` servlet for the three possibilities it handles—a nonvote ballot, a second ballot from the same client, and a ballot with a legitimate vote—are shown in Figures 10.6, 10.7, and 10.8.

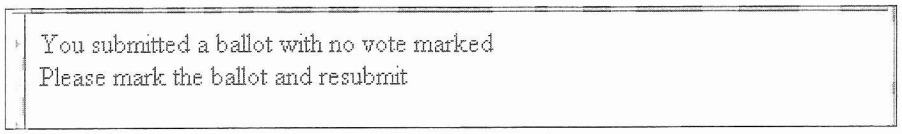


Figure 10.6 The output of the `VoteCounter` servlet for a form with no vote

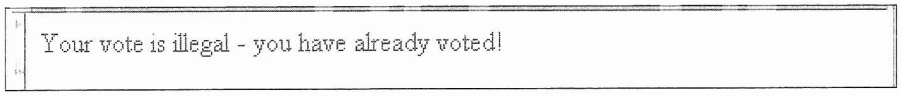


Figure 10.7 The output of `VoteCounter` for a form with a second vote from the same client

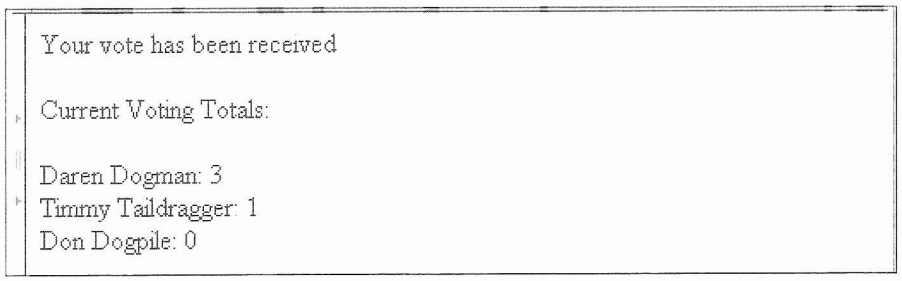


Figure 10.8 The output of `VoteCounter` for a form with a legitimate vote

10.4.4 Session Tracking

Recall that a session is a sequence of HTTP communications between one specific browser and one specific server. Sessions are usually terminated either when the browser is connected to a different server or a server's chosen session time limit is reached.

In many cases, information about a session is needed only during the session. Also, the needed information about a client is nothing more than a unique identifier for the session, which is commonly used in shopping cart applications. For these cases, Java supports a simple alternative to using cookies directly for storing information about requests made by a client during a session; it is called *session tracking*. Session tracking is done using an `HttpSession` object, often called a *session object*, which can store a list of names and values that are similar to cookies. So, rather than using one or more cookies, a single session object can be used to store information about the previous requests of a client during a session. In particular, session objects often store a unique session ID for a session. One signficant way that session objects differ from cookies is that they can be stored on the server, whereas cookies are stored on the client. Session tracking also can be implemented by a servlet container with cookies, in which case the session information is actually stored on the client. The most useful methods for session objects are described in the following paragraphs.

The `putValue` method takes two objects, `String` and `Object`. The `String` object represents the name of a value; the `Object` object is the value to be bound to the name. Both the name and the value are stored in the session object on which `putValue` is called. For example, if there is a session named `mySession`, we could have the following:

```
mySession.putValue("iVoted", "true");
```

The `invalidate` method takes no parameters. It invalidates or destroys the session on which it is called.

The `removeValue` method takes a `String` parameter, which it interprets as the name of a value in the session object. The action of `removeValue` is to delete the data bound to the given name.

The `getValue` method takes a `String` parameter, which it interprets as the name of a value in the session object. `getValue` returns a reference to the value bound to the given name. The value returned by `getValue` is often cast so that it can be used as a reference to a particular type. For example, consider the following statement:

```
String theValue = mySession.getValue("iVoted");
```

The `getValueNames` method takes no parameters. It returns the names of all values in the session object as an array of strings:

```
String valueNames [] = mySession.getValueNames();
```

A client request that includes a session object has that object attached to the `HttpServletRequest` object. To access such a session object, we use the `getSession` method of `HttpServletRequest`, which takes a `Boolean` object as a parameter. If the value `true` is given as a parameter and no session object is attached to the `HttpServletRequest` object, one is created and a reference to it is returned. If the value `false` is given as a parameter and there is no session object, `getSession` returns `null`.

The use of session tracking is illustrated with an alternative vote-counting servlet named `VoteCounter2`. The purpose of this servlet is the same as the earlier vote-counting servlet, which used cookies to disallow multiple votes from the same client. This example is slightly less realistic because it uses session tracking to check for multiple votes. This is unrealistic because the user can simply exit the browser, reenter the browser, which creates a new session, and vote again. In the case of the cookie voting control, the user was disallowed multiple votes for a specified period of time, regardless of how many times he or she exited and reentered the browser.[2] The following is the code for the session-tracking vote counter:

```
// VoteCounter2.java
// This servlet processes the ballot form, returning a
//  page asking for a new vote if no vote was made on the
//  ballot. For legitimate ballots, the vote is added to
//  the current totals, and those totals are presented to
//  the user in a return page.
// This servlet uses session tracking rather than cookies.
// The voting data file, votesdat.dat, is stored on the Web server.

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class VoteCounter2 extends HttpServlet {
    PrintWriter servletOut;
    int index;
    public void doPost(HttpServletRequest request,
                       HttpServletResponse response)
        throws ServletException, IOException {
        HttpSession mySession = null;
        int votes[] = null;
        String vote;
        File votesdat = new File("votesdat.dat");
        String candidates[] = {"Daren Dogman", "Timmy Taildragger",
                               "Don Dogpile"};

// Check to see if there was a vote on the form
        vote = request.getParameter("vote");
        if (vote == null) {  // There was no vote
```

2. Of course, it is easy to cheat our cookies voting system. The user could set his or her browser to not accept cookies or could simply delete them after voting. Using a different machine for each vote would also work.

```
// Create the return page
            makeHeader(response);
            servletOut.println(
                "You submitted a ballot with no vote marked");
            servletOut.println(
                "Please mark the ballot and resubmit");
        }  // end of if vote == null) ...
        else {  // There was a vote

// Check to see if this client voted before
            if (!votedBefore(mySession, request)) {

// No previous vote, so read the file (if it exists)
// Synchronize a block for the file access
                synchronized(this) {
                    if (votesdat.exists()) {
                        ObjectInputStream indat =
                            new ObjectInputStream(
                            new FileInputStream(votesdat));

// We need try/catch here because readObject may throw the
//  ClassNotFoundException
                        try {
                            votes = (int []) indat.readObject();
                        }
                        catch(ClassNotFoundException problem) {
                            problem.printStackTrace();
                        }
                        indat.close();
                    }  // end of if (votesdat...

// If the file does not exist (this is the first vote), create the
//  votes array
                    else
                        votes = new int[3];

// Add the new vote of the response to the votes array
                    if (vote.equals("Dogman"))
                        votes[0]++;
                    else if (vote.equals("Taildragger"))
                        votes[1]++;
                    else votes[2]++;

// Write updated votes array to disk
                    ObjectOutputStream outdat =
                        new ObjectOutputStream(
```

```
                        new FileOutputStream(votesdat));
                    outdat.writeObject(votes);
                    outdat.flush();
                    outdat.close();
                } //** end of the synchronized block

// Create a session object and set a value to indicate a vote
                mySession = request.getSession(true);
                mySession.putValue("iVoted", "true");

// Write a response message
                makeHeader(response);
                servletOut.println("Your vote has been received");
                servletOut.println(
                    "<br/> <br/> Current Voting Totals:<br/>");

// Create the total votes return information
                for (index = 0; index < 3; index++) {
                    servletOut.println("<br/>");
                    servletOut.print(candidates[index]);
                    servletOut.print(": ");
                    servletOut.println(votes[index]);
                }
            } // end of if (!votedBefore( ...
            else {  // The client voted before

 // Write a response message
                makeHeader(response);
                servletOut.println(
                    "Your vote is illegal - you already voted!");
            } // end of else
        } // end of else (there was a vote)

        servletOut.println("</body> </html>");
        servletOut.close();

    } // end of doPost
//-----------------------------------------------------------------
// Method votedBefore - return true if the client voted before;
//  false otherwise
    boolean votedBefore(
                HttpSession mySession, HttpServletRequest request) {

// Get the session object, if there is one
        mySession = request.getSession(false);
```

```
// If there was no session, the vote must be okay
        if (mySession == null)
            return false;
        else { // there was a session
            String names [] = mySession.getValueNames();
            for (index = 0; index < names.length; index++) {
                if (names[index].equals("iVoted") &&
                 mySession.getValue(names[index]).equals("true"))
                    return true;
            }  // end of for (index = 0; ...
            return false;
        }  // end of else
    }  // end of votedBefore
//-----------------------------------------------------------------
// Method makeHeader -
// get the writer and produce the response header
    void makeHeader(HttpServletResponse response)
            throws IOException {

// Set content type for response and get a writer
        response.setContentType("text/html");
        servletOut = response.getWriter();

// Write the response document head and the message
        servletOut.println("<html><head>");
        servletOut.println(
            "<title> Return message - </title></head><body>");
    }  // end of makeHeader
  }  // end of VoteCounter2
```

10.5 Java Server Pages

Java Server Pages (JSP), which are built on top of servlets, provide alternative ways of constructing dynamic Web documents. It is "ways," not "way," because JSP includes several different approaches to dynamically generate Web documents. JSP, which was designed by Sun Microsystems, is a specification rather than a product. This means that Sun does not provide code to support JSP. Other organizations are invited to implement the specification. This encourages competition among providers, which can result in better quality software.

10.5.1 Motivations for JSP

There are several perceived problems with the servlet approach, as well as other related approaches, to providing dynamic Web documents. Among these are the

problem of having the XHTML response document embedded in programming code. In the case of servlets, the entire response document is created by calls to the `println` method of the `PrintWriter` class. This forces all maintenance of the user interface of the document to be done on program code.

A closely related problem is that organizations often have two different kinds of personnel, with different skill sets, work on the development and maintenance of Web documents. Web designers focus on interface and presentation characteristics of Web documents. Programmers, on the other hand, design and maintain the code that processes form data and handles interactions with databases. Most personnel belong in one or the other of these categories rather than both. Yet having XHTML code and programming code intermixed requires people from both categories to work on the same documents. Furthermore, these mixed-code documents are difficult for people from both categories to read.

JSP takes an opposite approach to that of servlets: Instead of embedding XHTML in Java code that provides dynamic documents, code of some form is embedded in XHTML documents to provide the dynamic parts of a document. These different forms of code are what make up the different approaches used by JSP.

The basic capabilities of servlets and JSP are the same. The basis for deciding which to use is explained in Section 10.5.2.

10.5.2 JSP Documents

There are two syntactic forms for writing JSP documents, the original way, now called *classic syntax*, and the alternative way, which uses XML syntax. The XML approach became possible in JSP 2.0, which was released in late 2003. The XML syntax of JSP documents is not discussed in this chapter and all of our JSP document examples in this book use classic syntax.

When requested by a browser, a JSP document is processed by a software system called a JSP container. Some JSP containers compile the document when the document is loaded on the server; others compile them only when they are requested. The compilation process translates a JSP document into a servlet and then compiles the servlet. So, JSP is actually a simplified approach to writing servlets.

A JSP document consists of four different kinds of elements: traditional XHTML or XML code, action elements, directives, and scriptlets.

The XHTML or XML code in the document is used to produce the content that is fixed. This markup is called *template text*. It is the static part of the document. Everything in a JSP document that is not a JSP element is template text. Template text is not modified by the JSP containers—it arrives at the browser exactly as it appears in the JSP document. The designer choice between using a servlet and a JSP document is made on the basis of the proportion of the document that is template text. The more template text there is, the better it is to use JSP. If a document is mostly dynamically generated, then a servlet is the better choice.

Action elements dynamically create content. The document that results from the execution of a servlet whose source is a JSP document is a combination of the template text and the output of the action elements. An action element has the form of an XHTML element: an opening tag, possibly including attributes; content, which is sometimes called the *action body*; and a closing tag. In fact, however, action elements represent program code that generates XHTML markup.

Action elements appear in three different categories: standard, custom, and JSP Standard Tag Library (JSTL). The standard action elements are defined by the JSP specification. These include elements for dealing with Java beans,[3] including the response from a servlet or another JSP document, and dynamically generating an XML element. For example, the action element `<jsp: element>` dynamically generates an XML element, possibly with attributes and content defined by other nested actions. The `<jsp:include>` action element specifies a document file as the value of its `page` attribute. The document file is copied into the output document of the JSP document in which the `include` appears.

Custom action elements are those that are designed for a specific category of JSP documents within an organization. Because of its complexity, the development of custom action elements is not discussed in this chapter.

The JSP standard action elements are highly limited in scope and utility, so there are many commonly needed tasks that cannot be done with them. This led to a large number of different programmers defining their own custom action elements for these tasks, which was clearly a waste of effort. This situation was remedied by the development of the JSTL, which includes action elements for many commonly needed tasks. The JSTL actually consists of five libraries. The *Core* library includes elements for simple flow control, in particular, selection and loop constructs, among others. The *XML Processing* library includes elements for transformations of XML documents, including those specified by XSLT style sheet documents. The *Internationalization and Formatting* library includes elements for formatting and parsing localized information. The *Relational Database Access* library includes elements for database access. The *Functions* library includes elements for Expression Language functions. (Expression Language is described in Section 10.5.4.)

Action elements specify actions that are described with statements in a programming language. In fact, libraries of action elements form programming languages that can be used to write dynamic actions in a markup-like form. The difference between using the action elements and using Java is twofold: First, the syntax is completely different. Second, the special tags are simpler and easier to use than their Java equivalents.

A directive is a message to the JSP container, providing information about the document and the sources of predefined action elements of the document.

3. A Java bean is a reusable component. They are beyond the scope of this book and will not be discussed here.

Directives can specify that content from other sources be included in a document. However, directives do not themselves produce content.

Syntactically, directives are tags that use `<%@` and `%>` delimiters. They use attributes to communicate to the container. The most commonly used directives are `page` and `taglib`. The `page` directive can include many different attributes, but only one is required, `contentType`, which is usually set to `text/html`, as in the following:

```
<%@ page contentType = "text/html" %>
```

The `taglib` directive is used to specify a library of action elements, or tags, that are used by the document. Many JSP documents use the JSTL. The URI for the JSTL is given as the value of the `uri` attribute in the `taglib` directive. Also included is the `prefix` attribute, which is assigned the abbreviation, or prefix, that the document uses for tags from the JSTL. For example, a JSP document may contain the following directive:

```
<%@ taglib prefix = "c"
  uri = "http://java.sun.com/jsp/jstl/core" %>
```

This directive specifies the URI of the JSTL Core library and sets the prefix for its elements to `c`. Examples of the use of Core library action elements appear in the JSP examples later in this chapter.

10.5.3 Scriptlets

Scriptlets are Java code scripts that can be embedded in JSP documents. When the JSP document is converted to a servlet, its scriptlets are simply copied into the servlet. There are four kinds of elements that can appear in a scriptlet: comments, scriptlet code, expressions, and declarations. The use of declarations can lead to threading problems, so they are rarely used. Because of this, they are not discussed here.

Scriptlet expressions are used to insert values into the response. The form of a scriptlet expression is as follows:

```
<%= expression %>
```

The expression is evaluated, and the result is converted to a `String` object (if it is not already a `String` object) and is placed in the response. Note that the expression cannot be terminated with a semicolon.

Scriptlet code is Java code that is delimited by `<%` and `%>`.

Scriptlet comments must be Java comments because scriptlets consist of Java code. This means that JSP comments (`<%-- ... --%>`) and XHTML comments, although both legal in JSP documents, are not legal in scriptlets.

Table 10.2 summarizes the four kinds of elements that can appear in a JSP document.

Table 10.2 JSP document elements

Element Kind	Purpose
XHTML	Describes the static part of the document
Action Elements	Create dynamic content
Directives	Communicate information about the document to the JSP container
Scriptlets	Compute and insert values into the response document, using Java code

The following is a simple example of an XHTML document that displays a form that collects a termperature in Celsius from the client. It then calls a JSP document that uses a scriptlet to compute and display the equivalent Fahrenheit temperature.

```
<!-- tempconvert0.html
     A document that displays a form that collects a Celsius
     temperature from a client and calls a scriptlet to
     convert it to Fahrenheit
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Get a Celsius temperature </title>
  </head>
  <body>
    <p>

<!-- Display a form to collect a Celsius temperature -->
       <form action = "tempconvert0.jsp" method = "get" >
         Celsius temperature:
         <input type = "text" name = "ctemp" />
         <input type = "submit" />
       </form>
     </p>
  </body>
</html>
```

The JSP document that is called by this document follows:

```
<!-- tempconvert0.jsp
     A document that converts a Celsius temperature received
     from tempconvert0.html to Fahrenheit
     -->
<html xmlns = "http://www.w3.org/1999/xhtml">
  <head>
    <title> Temperature converter </title>
  </head>
  <body>
    <p>
      <%

// Get the Celsius temperature from the form
         String strCtemp = request.getParameter("ctemp");
         float ftemp;

// convert the value to Fahrenheit
         ftemp = 1.8f * Integer.parseInt(strCtemp) + 32.0f;
      %>

<!-- Use an expression to display the value of the Fahrenheit
      temperature -->
     Fahrenheit temperature:
     <%= ftemp %>
  </body>
</html>
```

The XHTML and JSP documents for the temperature conversion can be combined into a single JSP document. This document has two parts: the XHTML markup to display the form to collect the Celsius temperature and the scriptlet to compute and display the Fahrenheit temperature. The document must first determine whether a request is the first or the second so that it can choose whether to display the form or compute and display the result. One way to do this is to use the `getParameter` method of the response object to get the form value and test it against the null string. If it is null, it is the first request, which requires that the form must be displayed. Otherwise, the computation must be done. The following is the combined document: